



PHM North America 2024 Challenge (Group A1)

Preventive Maintenance For Robotics And Intelligent Automation Course
Examination Project

Valerio Morelli
Federica Paganica
Federico Staffolani
Enrico Maria Sardellini
a.y. 2024/2025

PHM North America 2024 Challenge



EDA and Preprocessing

What kind of data will we be working with?



Probabilistic regression and fault detection

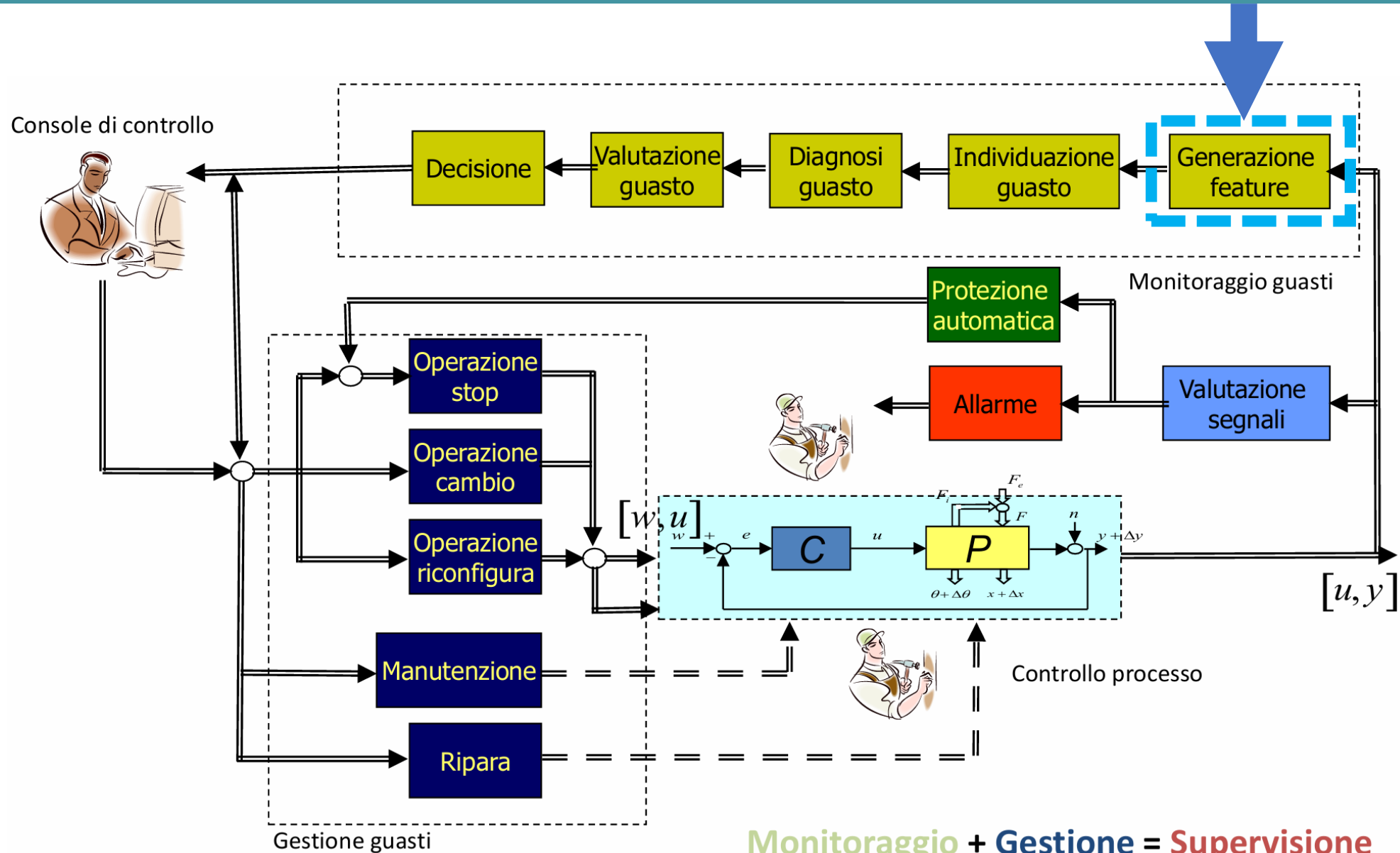
What are the results of our experiments?



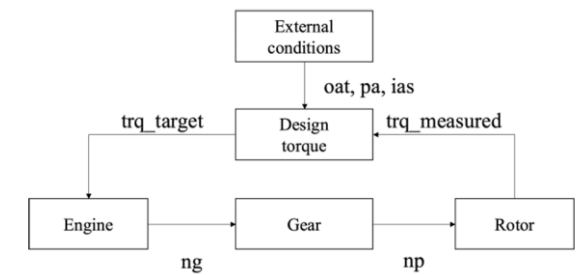
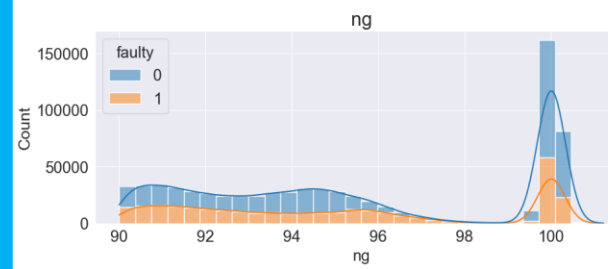
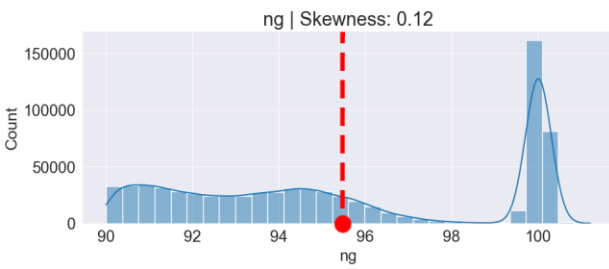
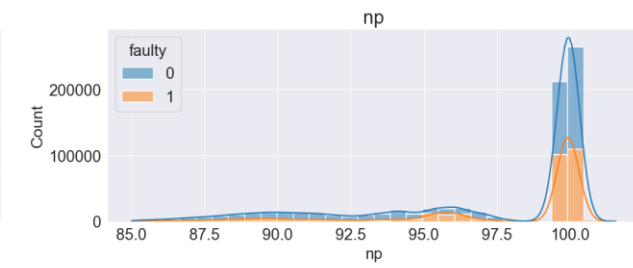
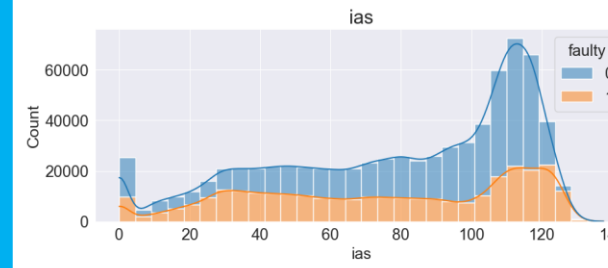
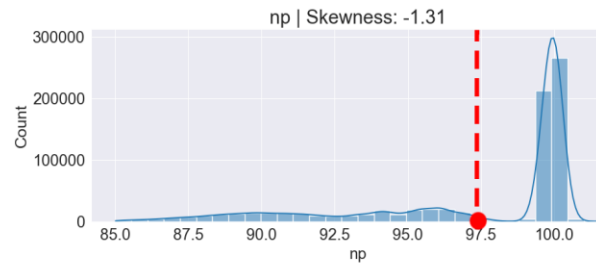
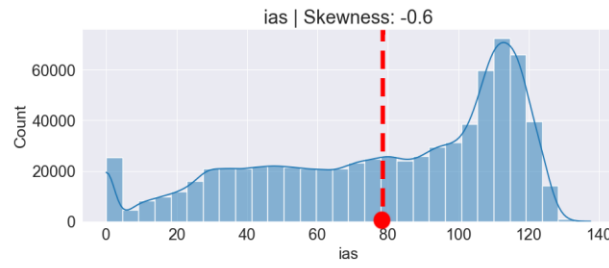
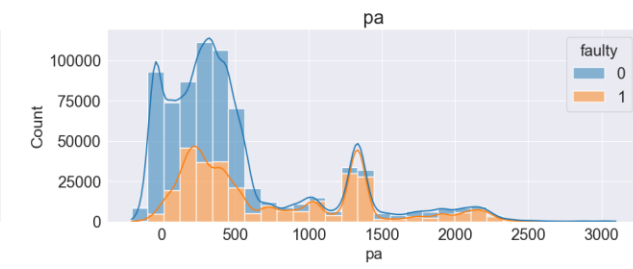
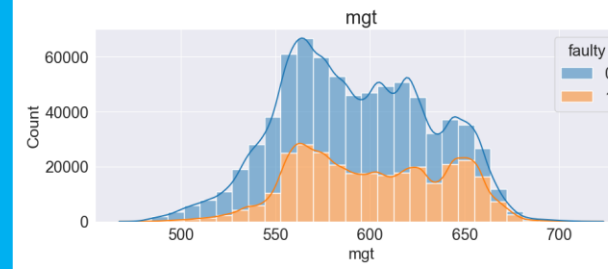
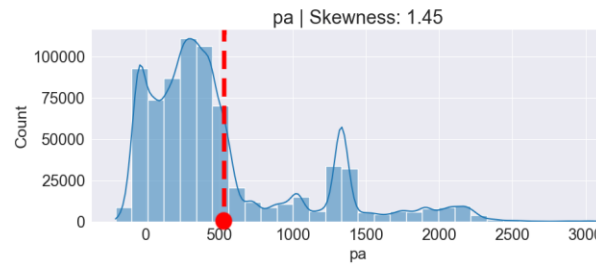
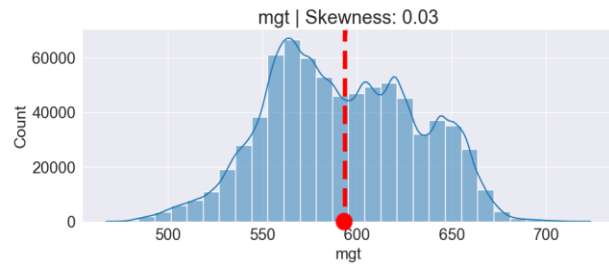
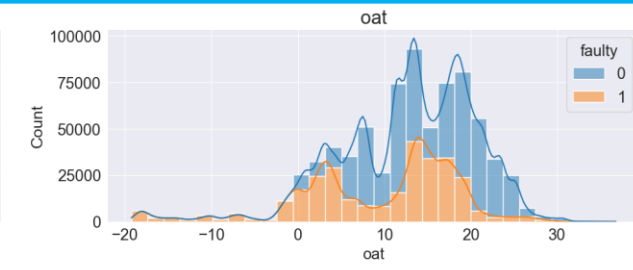
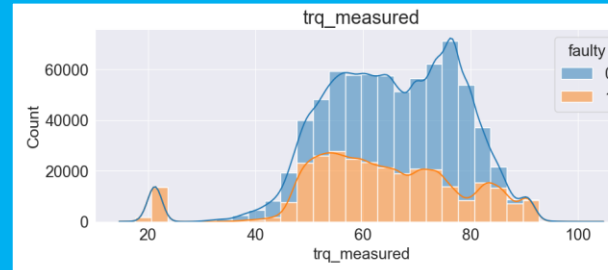
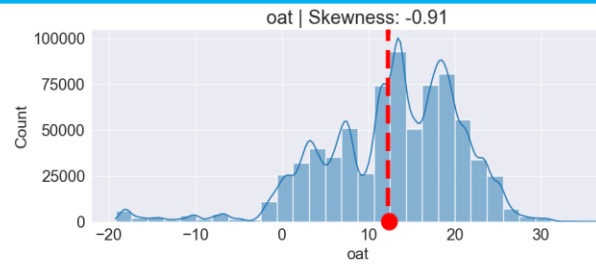
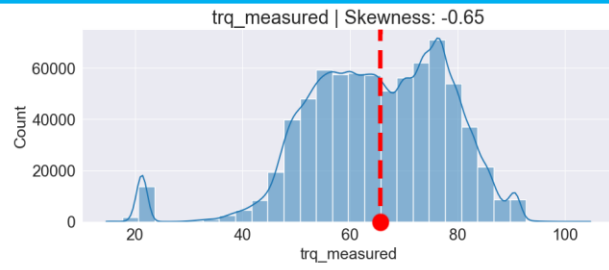
Next steps

What's missing?

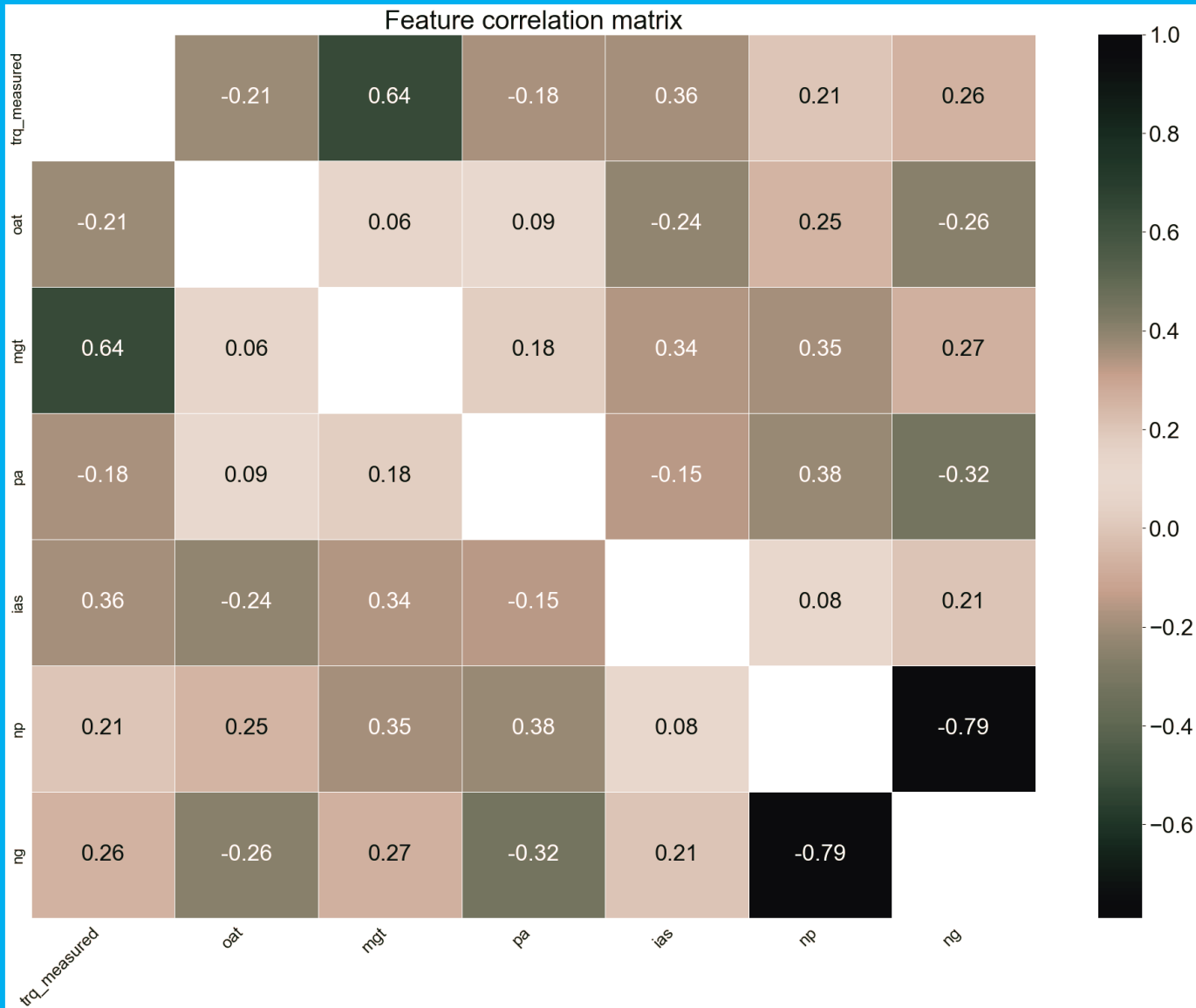
Where we are



Skewness study



Correlation matrix

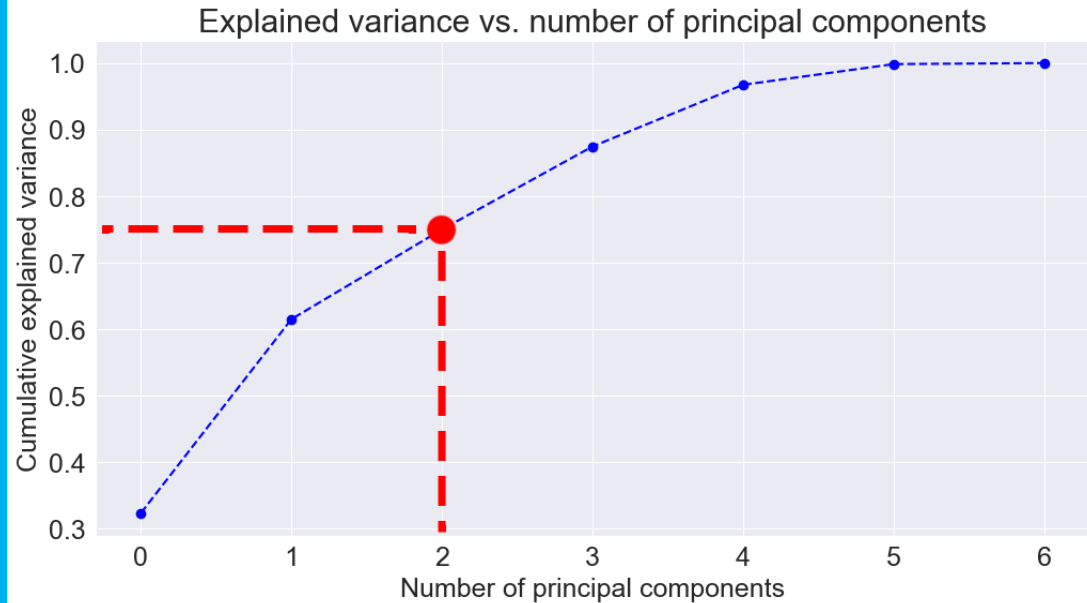


- ❑ From the correlation matrix, it is possible to verify how generally there is **very low correlation** between the features.
- ❑ The only exception of *np* and *ng* that have a negative correlation score of *0,79*. This corresponds to a **high eigenvalue** in the PCA.

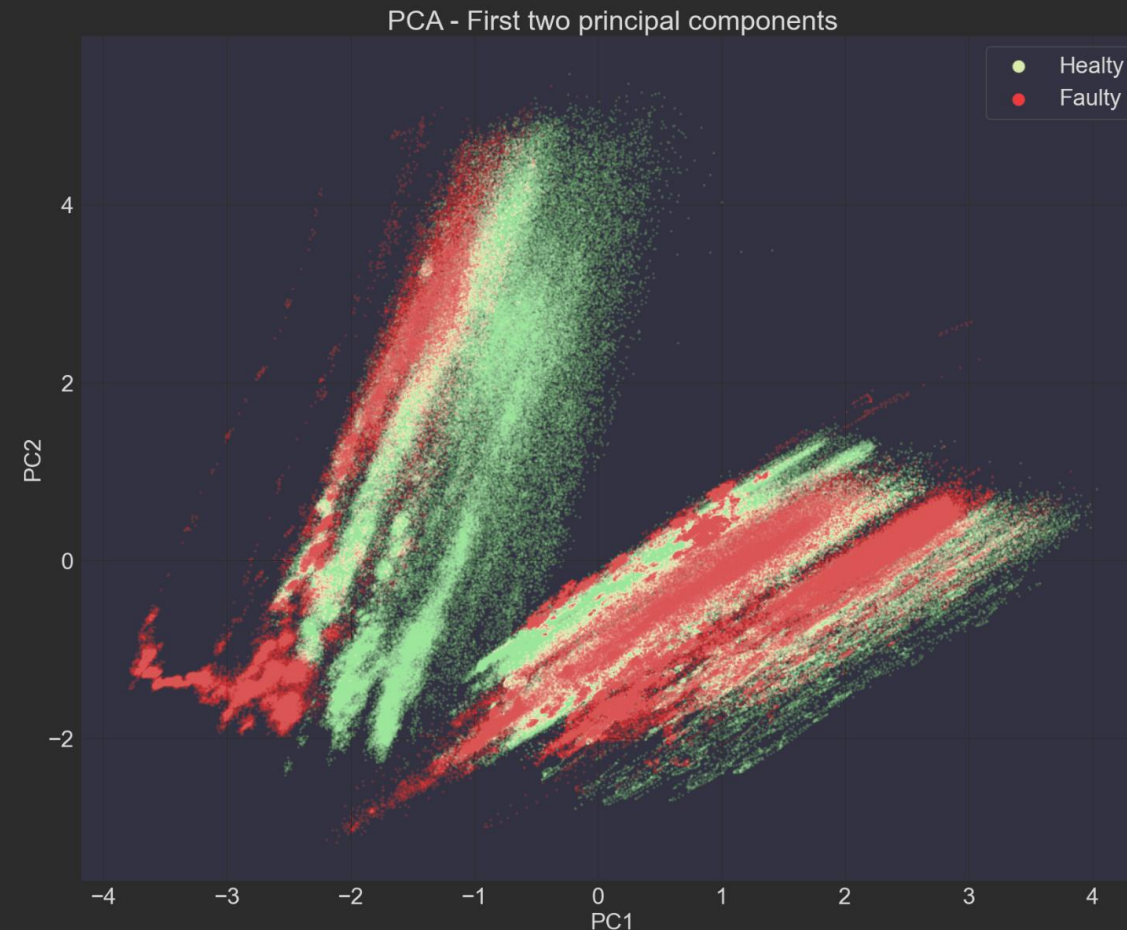
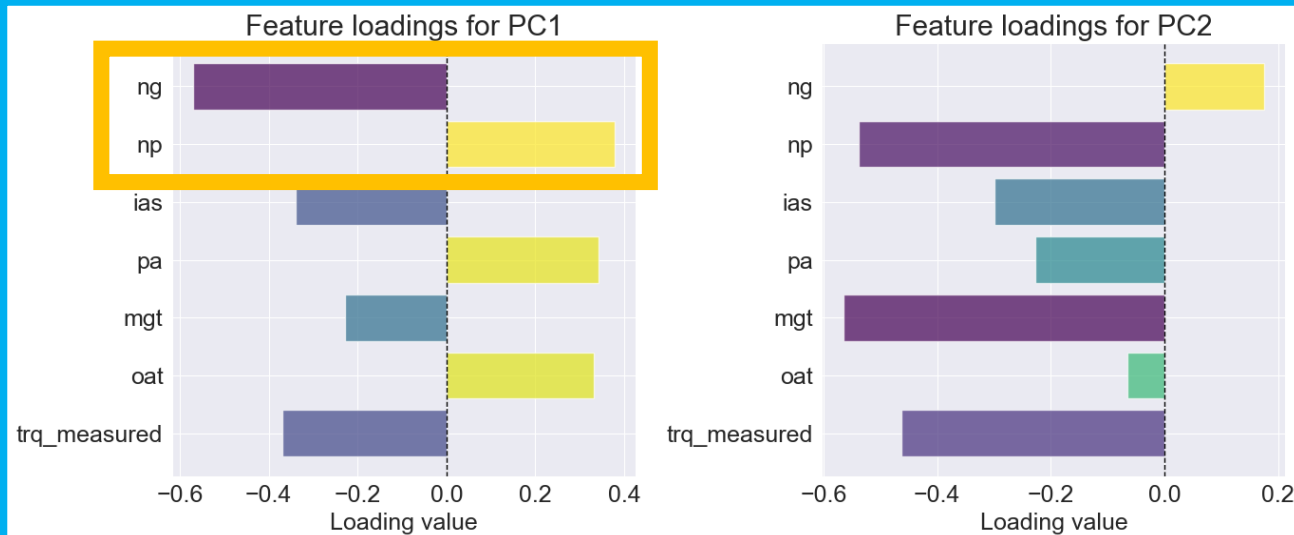


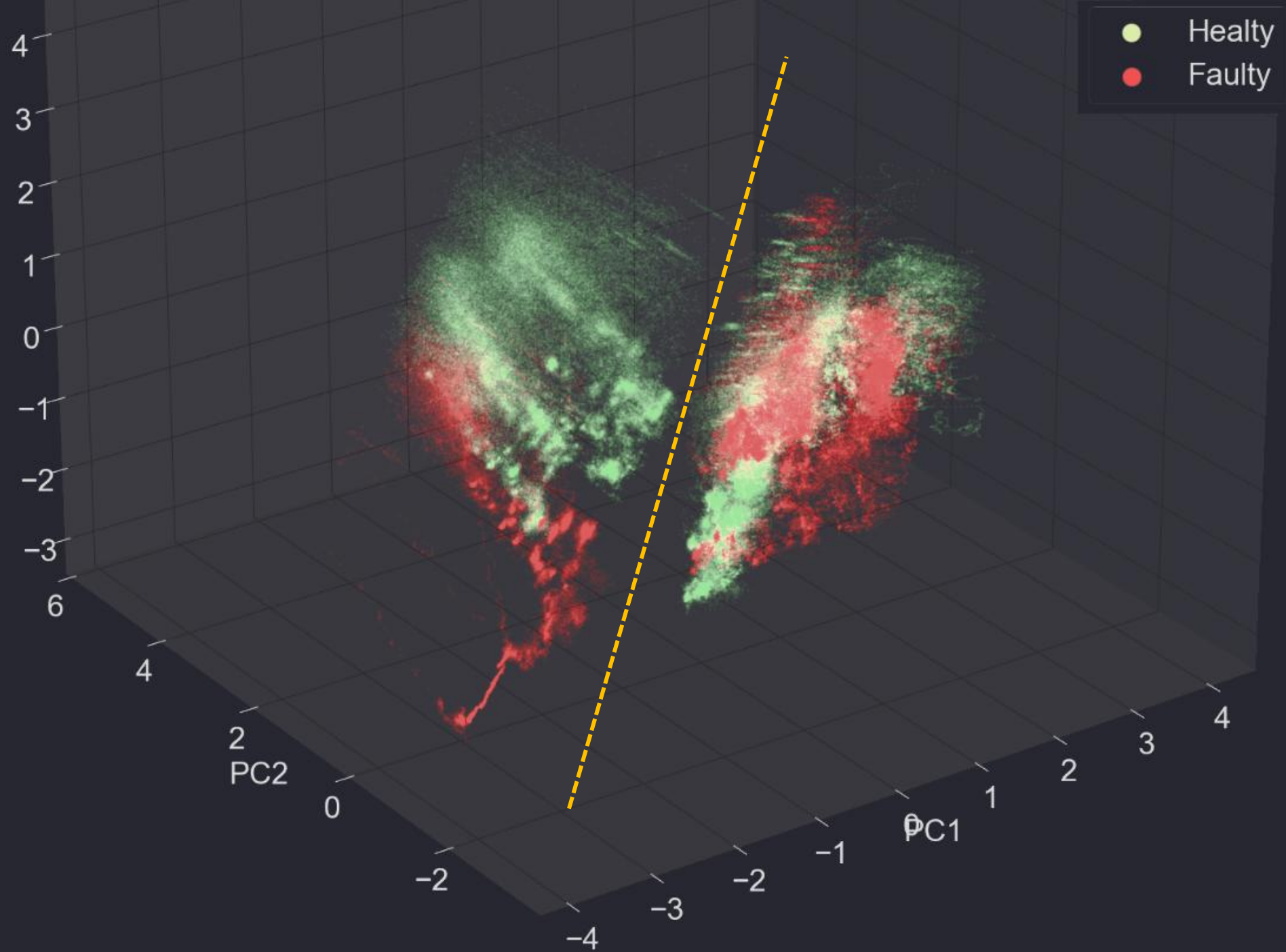
- ❑ For this reason, **we discarded the idea of merging one or more features** by eliminating individual ones

PCA analysis



- The PCA highlights 2 distinct clusters. As the paper "... proves, this is due to the *np/ng* ratio, the main loadings in the two directions.





Generating new features



- ❑ We created **polynomial features** up to **degree 3** using *PolynomialFeatures* from *sklearn*.

- ❑ Then **np/ng** was added as a new feature.

- ❑ We also normalized the original features by **dividing them by the density altitude** ρ .

- ❑ The **air density** is computed with the formula:

$$da = 1.2376 \times pa + 118.8 \times oat - 1782$$

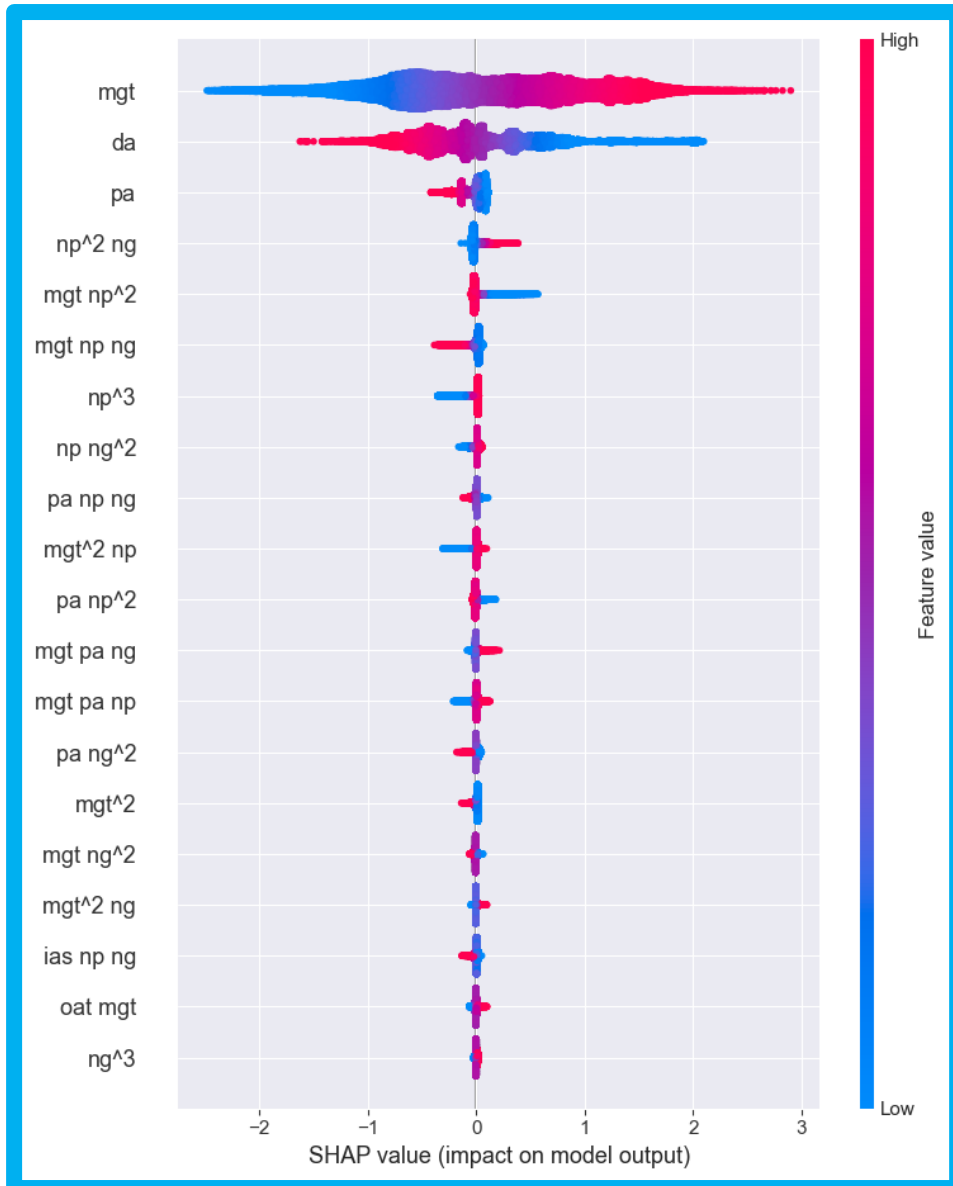
$$P = P_0 \times \left(1 - \frac{L \cdot h}{T_0}\right)^{\frac{g \cdot M}{R \cdot L}}$$

$$\rho = \frac{P \cdot M}{R \cdot T}$$

- ❑ **Two identical engines might operate differently** because one is in *denser* air and the other in *thinner* air $\Rightarrow$ each feature is divided by ρ . (*Normalization*)

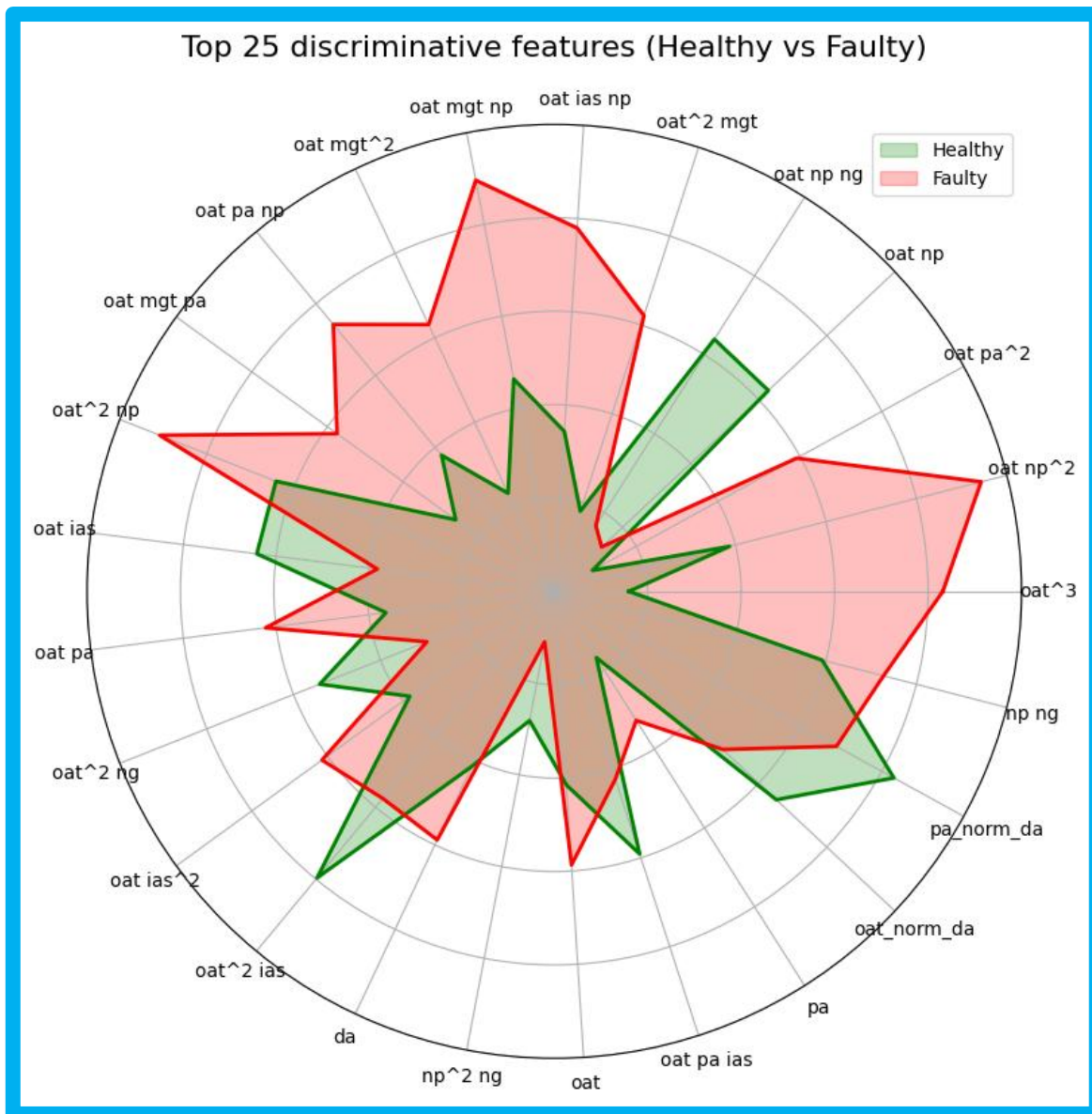
The total number of features obtained is 92, so we had to reason about **feature selection**, one for regression and one for classification task

Selecting the features to regress the torque target



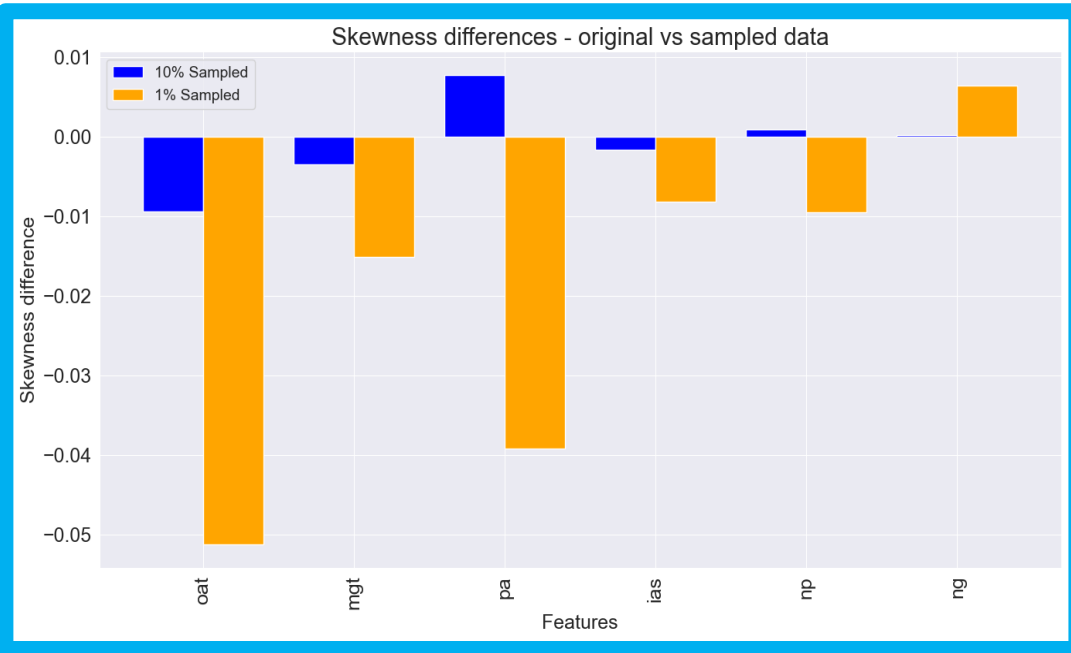
- ❑ We employed the **Shapley Value** to determine the **importance** of each feature in the torque target prediction.
- ❑ Red represents large values of a variable, and blue represents small ones. So, this plot can show a sense of the **direction of the impact** from the data on prediction.
- ❑ **Mgt** is the most important variable for this task, and generally it causes either a large positive or negative change in the prediction.
- ❑ For feature selection, we calculated the **mean absolute SHAP values** for each feature and selected the **top n most important features** based on these values (with $n=25$).

Selecting the features to train the fault detection classifier



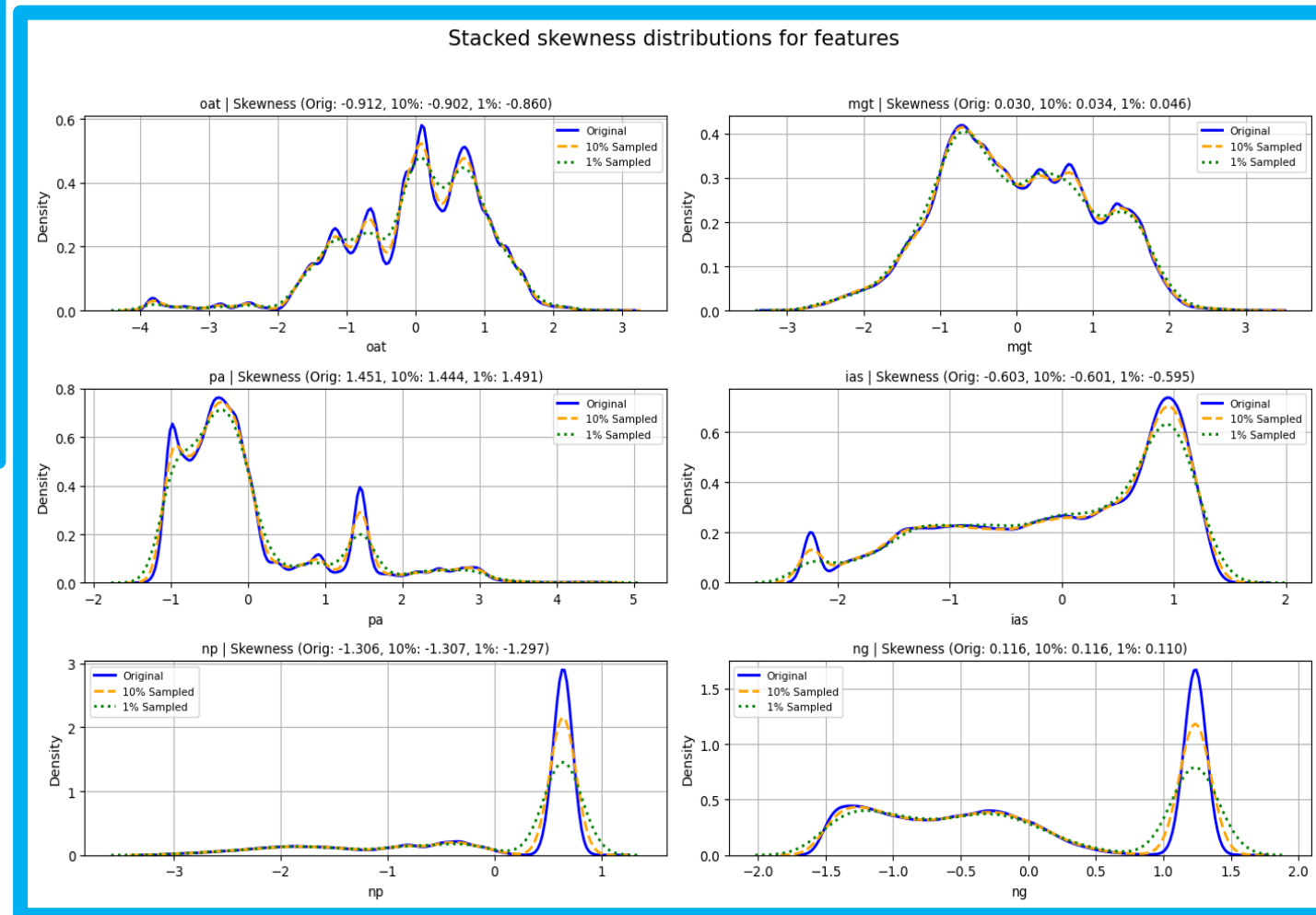
- ❑ For classification feature selection, we implemented a **radar chart (or spider chart)**.
- ❑ Data points are plotted along each axis according to their values. **Higher values** are plotted **further** from the center, while **lower values** are **closer** to the center.
- ❑ **Min-max scaling** was applied to normalize all features between 0 and 1 for a better visualization.
- ❑ Data was divided into two subsets: **healthy** (*faulty* == 0) and **faulty** (*faulty* == 1) subsets
- ❑ **To select the top n discriminant features, the absolute difference between the mean feature values for the two groups is used.**
- ❑ Also in this case was chosen n=25.

Justify the representativeness of a 10% subsample



❑ The skewness variation for the features in the dataset sampled at 10% is minimal.

❑ Due to the difficulty of handling the entire dataset for KAN, we have chosen this sampling method.



PHM North America 2024 Challenge



EDA and Preprocessing

What kind of data
will we be working
with?



Probabilistic regression and fault detection

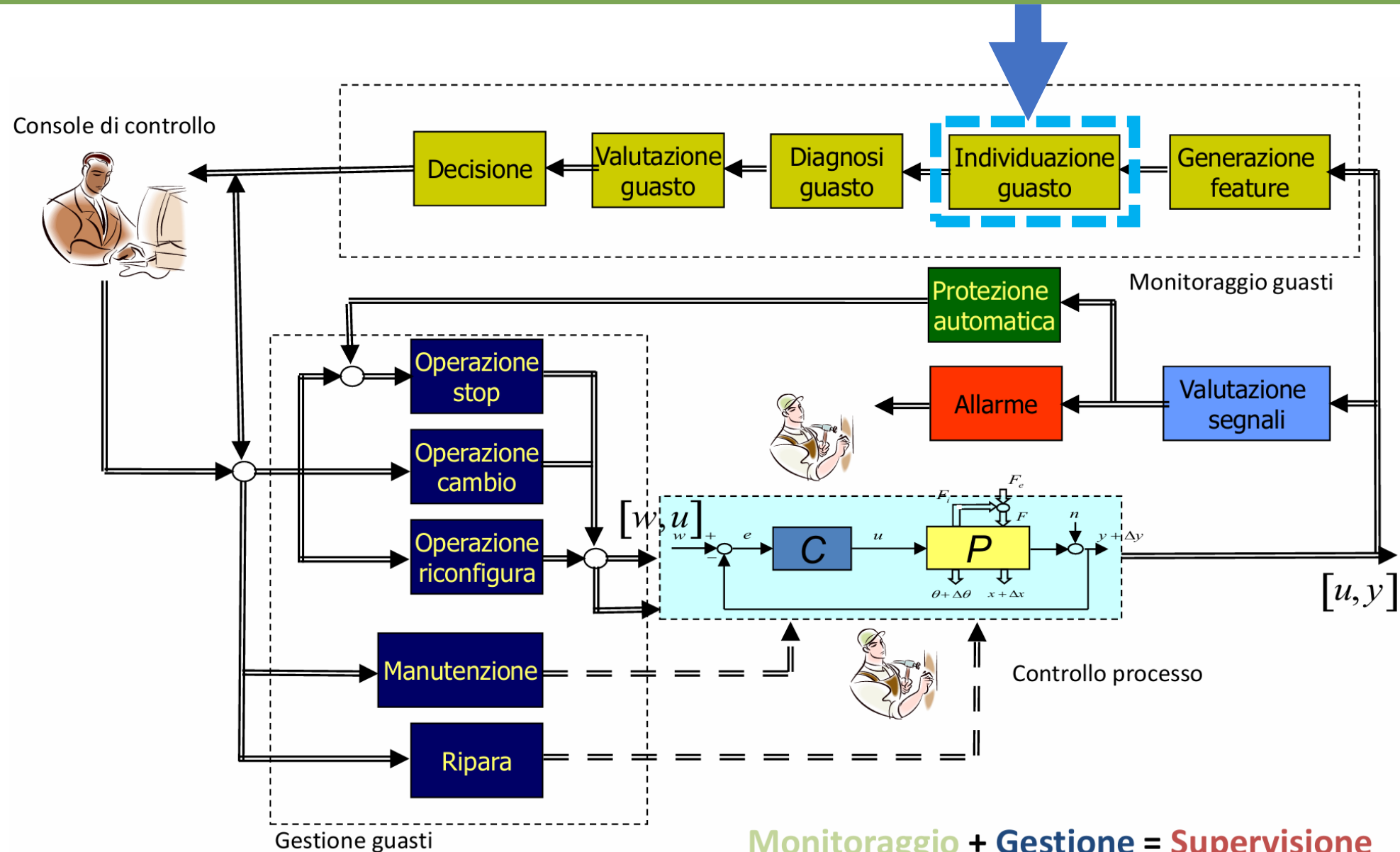
What are the results of
our experiments?



Next steps

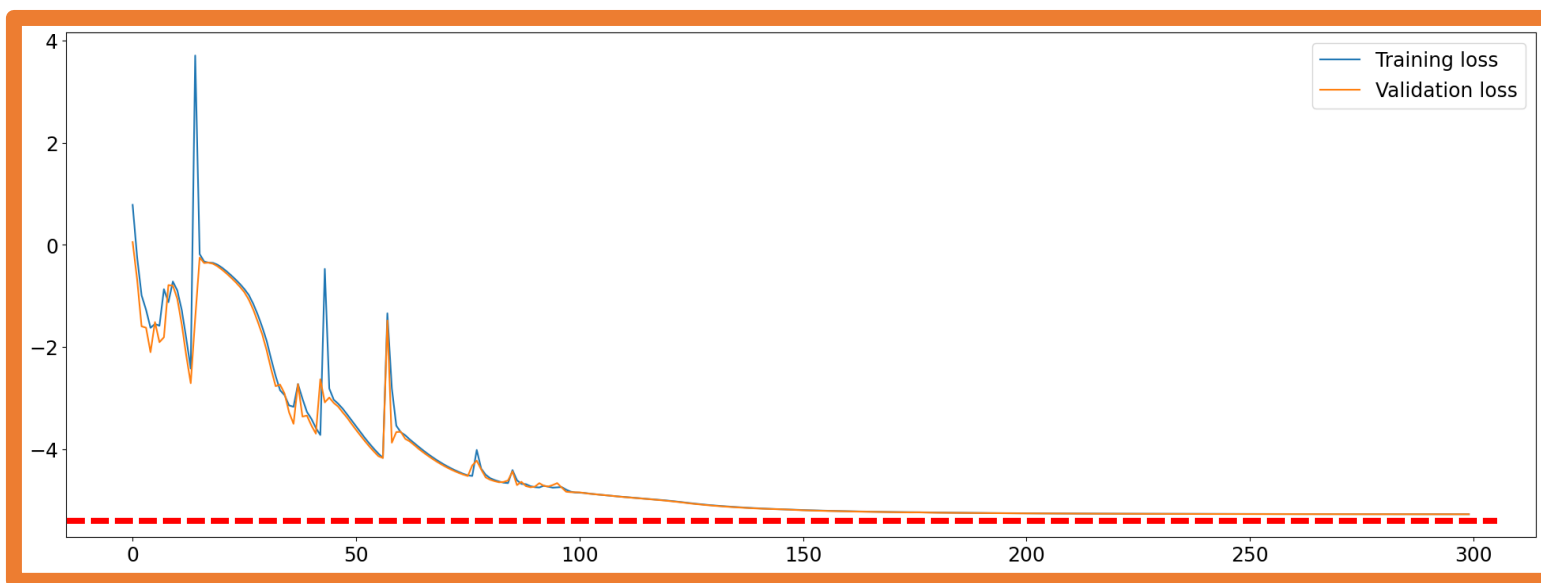
What's
missing?

Where we are



Probabilistic Regression with MLP

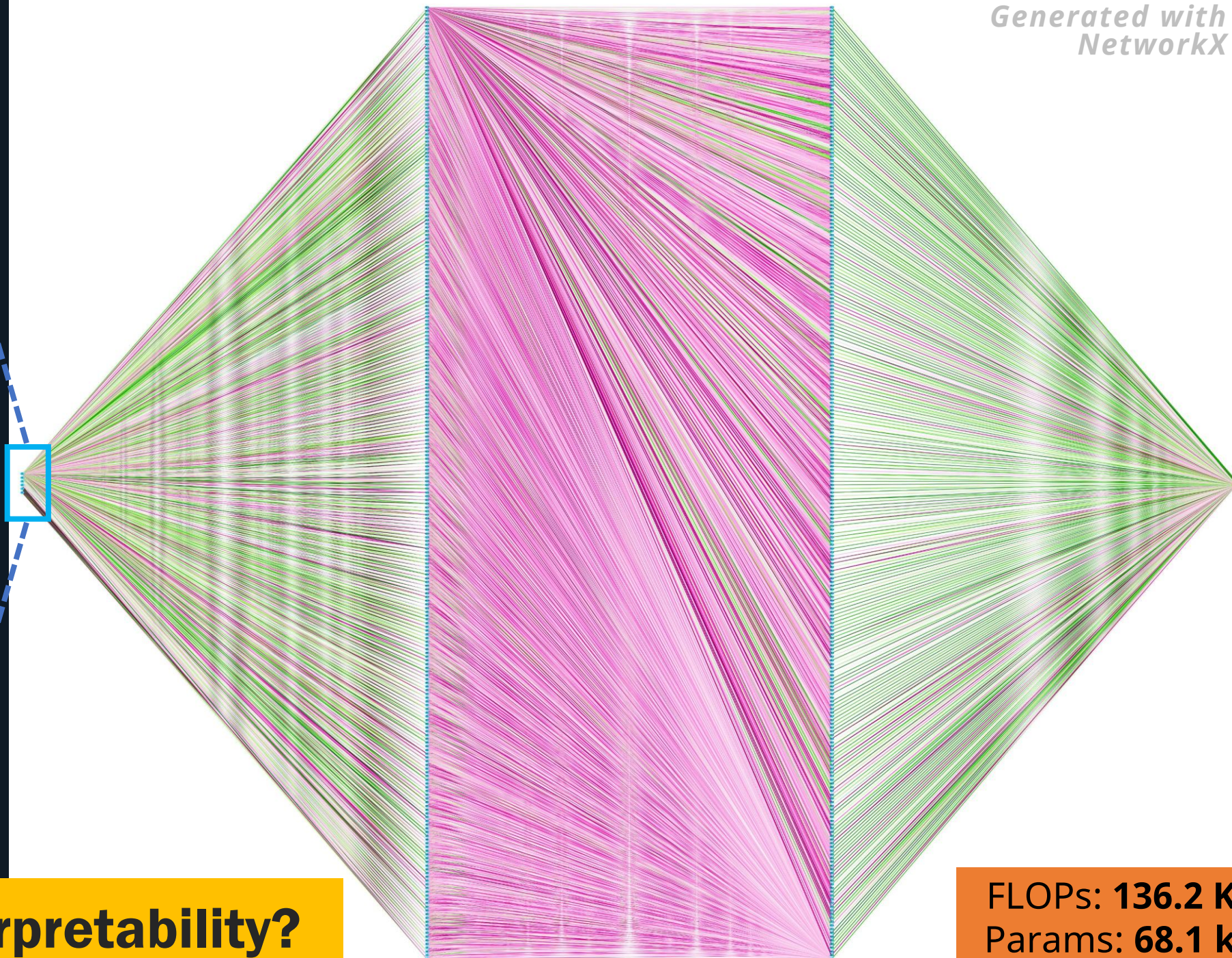
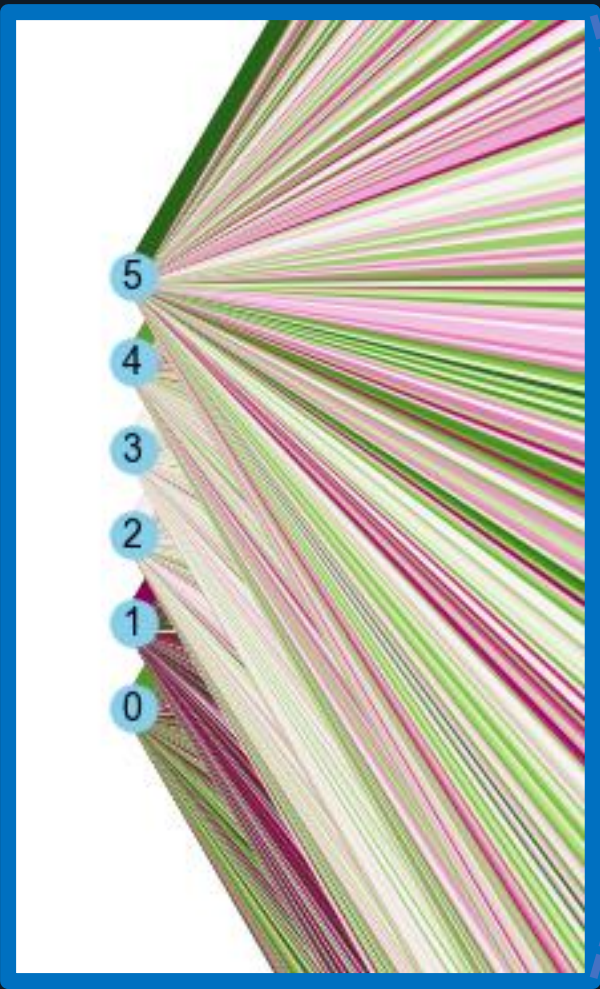
- ❑ The goal of the probabilistic regression is to find a **function that maps each set of input features to its own probability distribution**.
- ❑ The network outputs the parameters of the selected parametric PDF $\Rightarrow$ **Which PDF should we choose?**
- ❑ We assume that the underlying process has many small, independent disturbances. Thus, leveraging the **Central limit theorem**, we can say that the error between the model's prediction and **the actual observed value is likely to be normal**.



However...

- ❑ The paper "*Estimating ... Using a Probabilistic Neural Network*" proposes a **[7, 256, 256, 2, 1] MLP**.
- ❑ After training for 300 epochs, with **Gaussian Negative Log Likelihood Loss** the network reaches a huge accuracy, with an average answer variance of 1.26×10^{-5} .
- ❑ In other words, the model **can afford to hazard narrow confidence intervals**.

 Negative weight
 Positive weight



Generated with
NetworkX

...what about interpretability?

FLOPs: 136.2 K
Params: 68.1 k

The Kolmogorov-Arnold Theorem in ML

In 1957, **Arnold and Kolmogorov proved Hilbert's conjecture**, which stated that 7th-degree equations can't be solved using only two-variable functions, **wrong** with their KAT.

The KAT states that every multivariate continuous function $f : [0, 1]^n \rightarrow \mathbb{R}$, with $n \geq 2$, can be written as...

It is not an approximation like the UAT is.

$$f(\vec{x}) = \sum_{q=0}^{2n} \chi_q \left(\sum_{p=1}^n \psi_{p,q}(x_p) \right)$$

Outer functions

Inner functions

...namely as **a sum of composition of a finite number of *univariate* functions**
 $\Rightarrow$ ***The curse of dimensionality disappears!***

The KANs paper

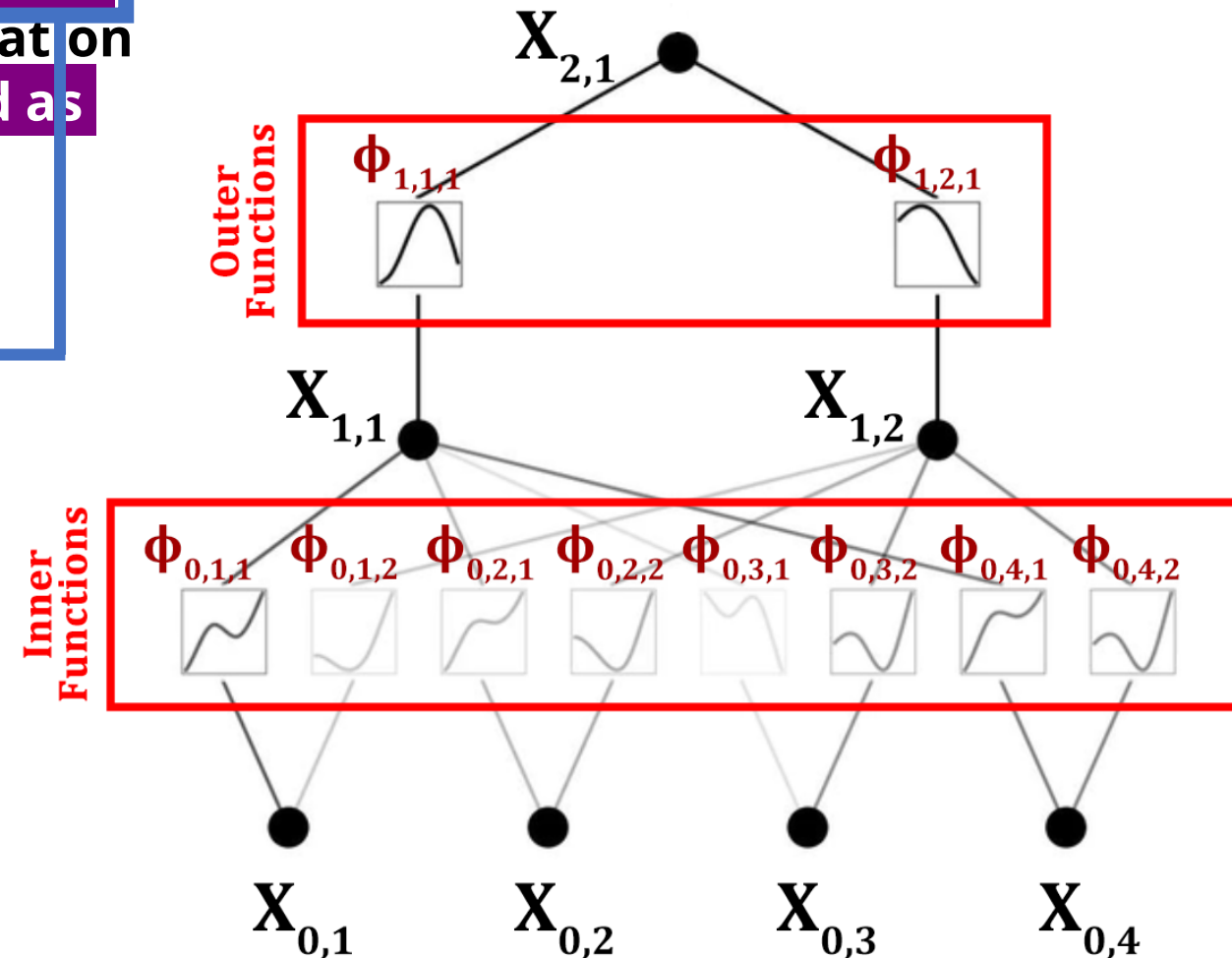
<< While MLPs **have fixed activation functions on nodes** ("neurons"), KANs have **learnable activation functions on edges** ("weights"), **parametrized as splines**. >> [Ziming Liu et al.]

Actually, MLPs also have learnable activation functions, such as *parametric ReLU*, *Swish* and the more recent *Maxout*

However, KANs already have an history in Machine Learning.

The **real contribution** of the paper lies in the **generalization** of the KAT in a **multi-layer** neural network of **arbitrary width**.

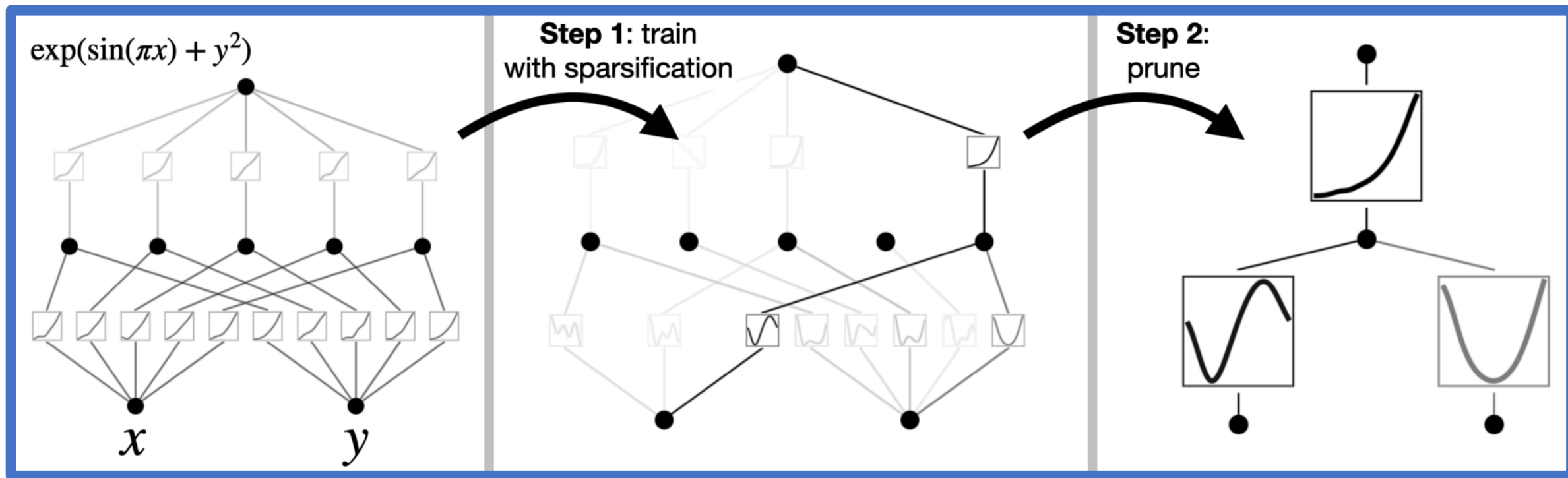
Stacking up multiple layers, KANs manage to reduce the well-known "**wild behaviour**" of the KAT functions.



The high interpretability of KAN

One of the aspects most emphasized in the paper is **the interpretability of KANs**.

After training, **they can be easily pruned**, to remove the activation functions below a certain significance threshold in exchange for a small error.

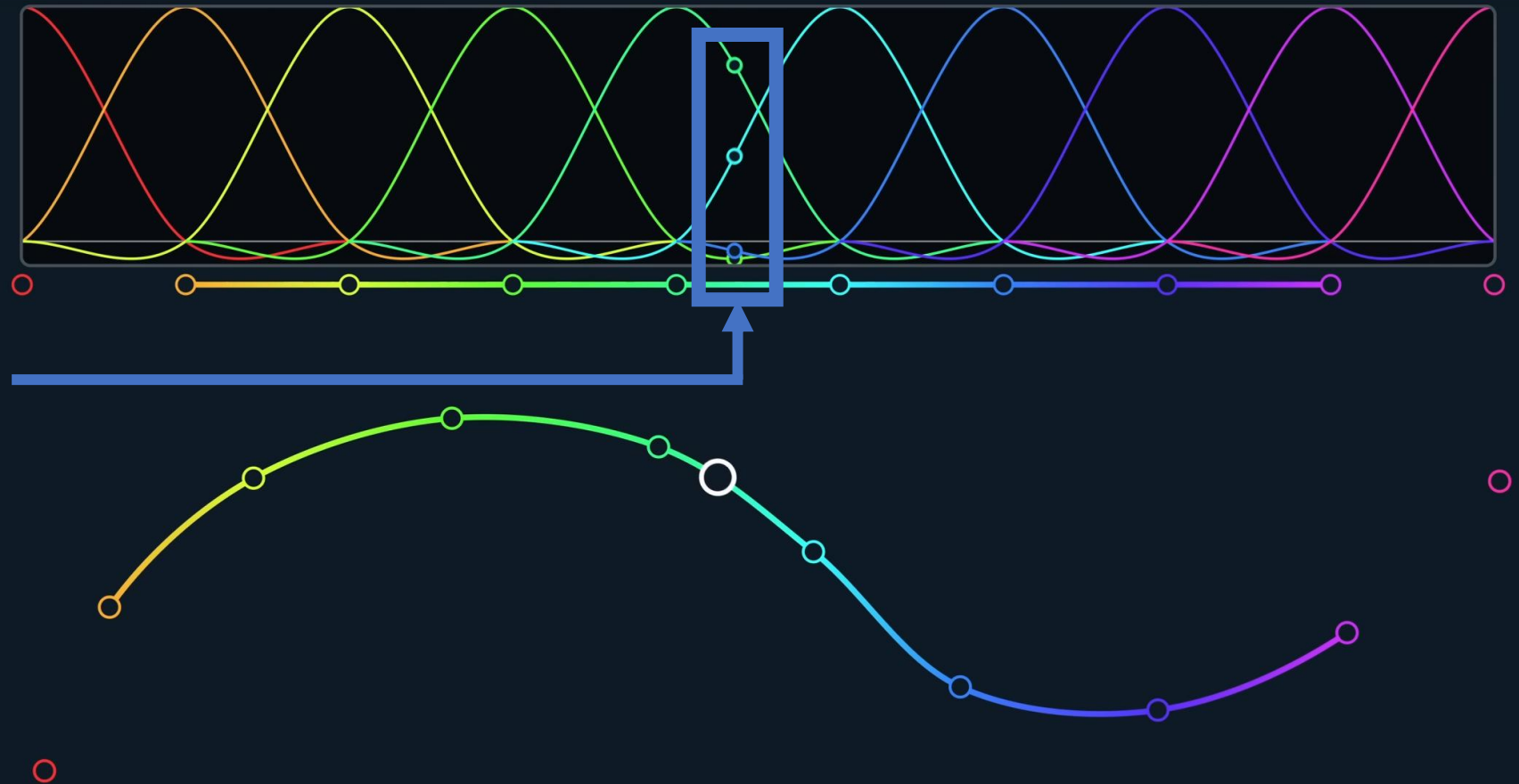


The splines

Splines are **curve generators**; they take some control points and produce a curve that interpolates (or not) between them.

The **basis functions** determine **how each point on the curve is affected by the control points**.

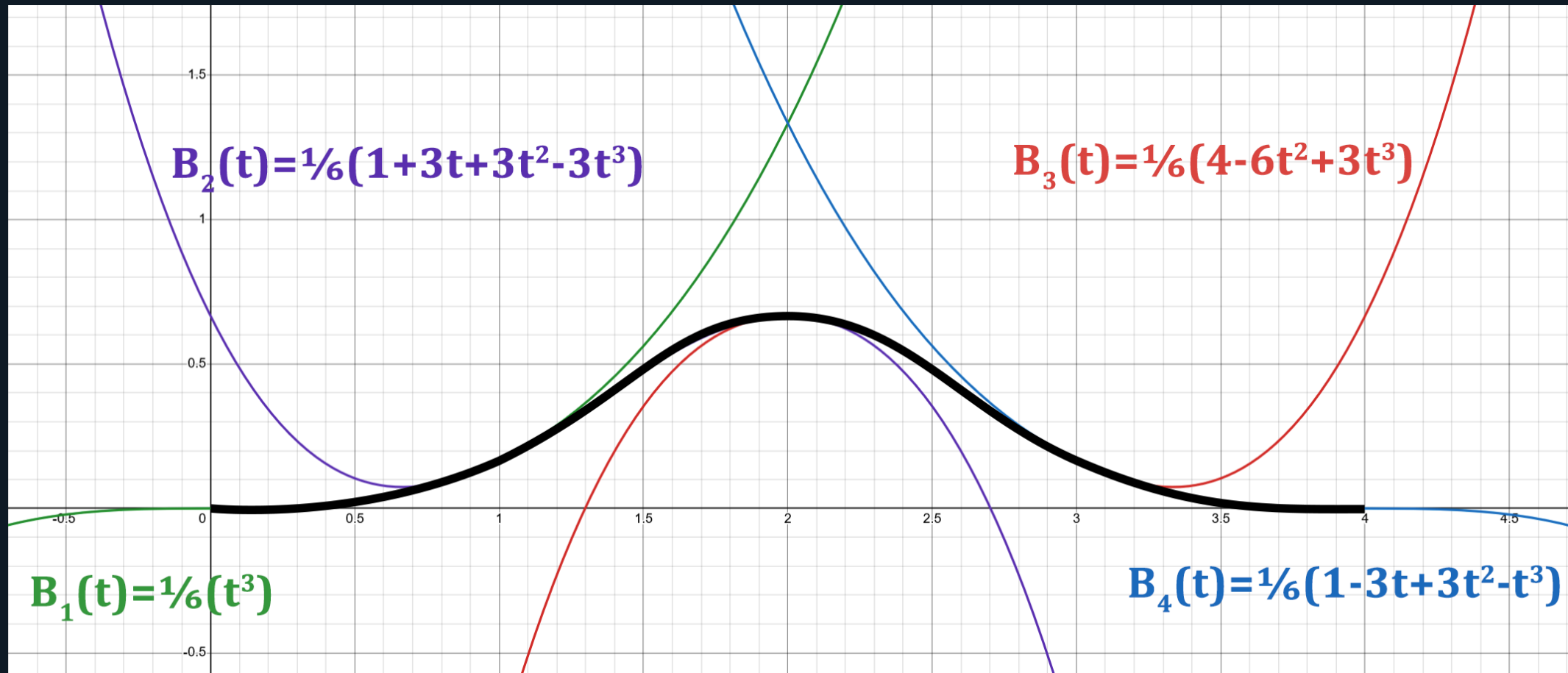
Each point is an **affine combination** of the control points. **The coefficients are given by the B-splines**



The B-Splines

The B-Spline equation is derived from the C^2 continuity constraints.

Thus, when it is convolved between the control points, it generates a C^2 B-form spline.



KANs choose B-form splines

The use of B-splines is supported by the findings of *Carl de Boor*, which proved that the **KAT** **remains a valid approximation** when B-form splines are employed.

The *residual*
component,
SiLU by default

The B-form spline,
defined by its $G + k$
control points

$$\phi(x) = w_b b(x) + w_s \sum_{i=1}^{G+k} c_i B_i(c) + \beta$$

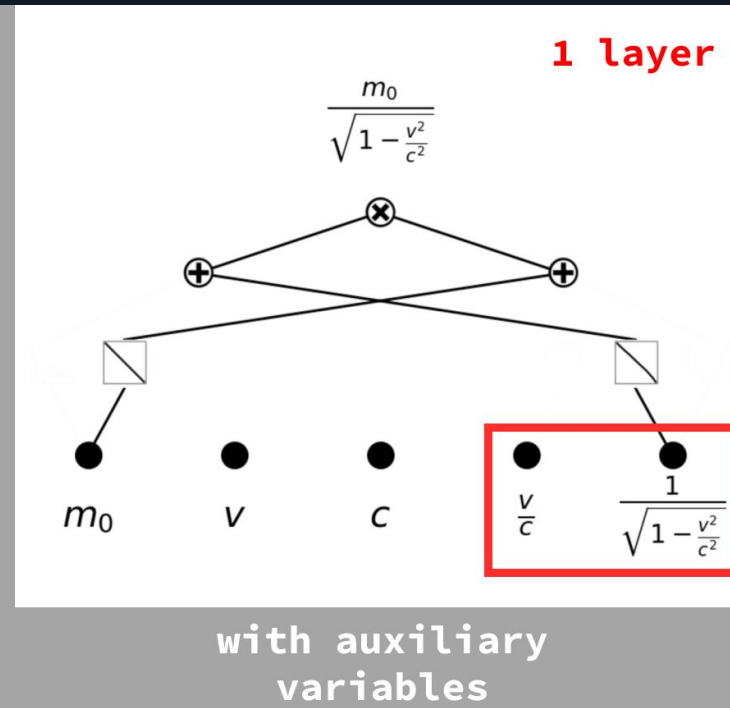
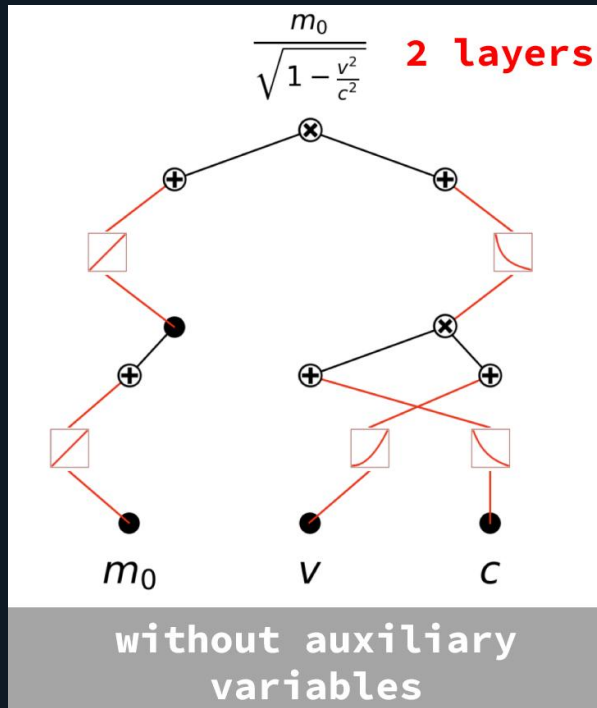
MultKANs as a generalization of KANs

On 19 August 2024, three and a half months after the introduction of KAN, the same authors published a paper on a generalization of KAN called **MultKAN**.

□ The main new feature introduced is the **multiplicative node**.

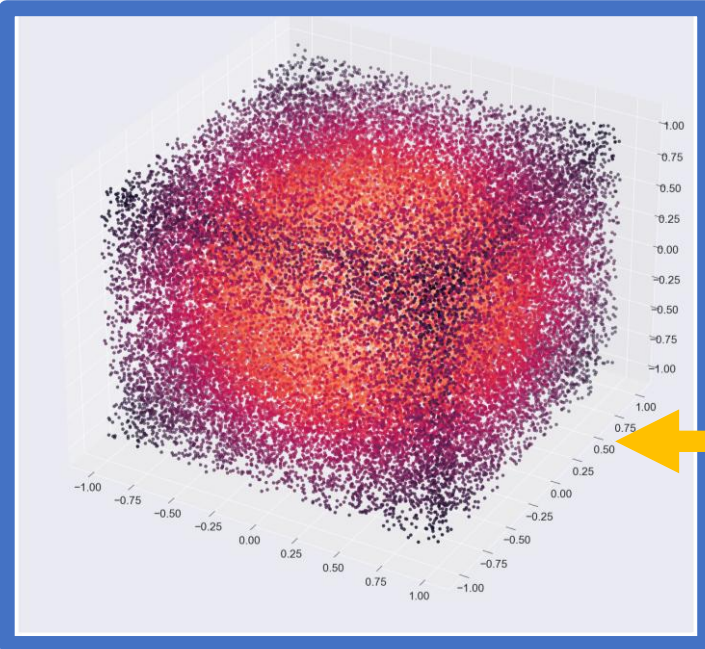
$$M_l(\mathbf{z}_l) = \underbrace{[\mathbf{z}_l^{(0)}, \mathbf{z}_l^{(1)}, \dots, \mathbf{z}_l^{(n_{l+1}^a)}]}_{\text{additive nodes}} \odot \underbrace{[\mathbf{z}_l^{(n_{l+1}^a+1)}, \dots, \mathbf{z}_l^{(n_{l+1}^a+n_{l+1}^m-1)}]}_{\text{multiplication nodes}} \odot \mathbf{z}_l^{(n_{l+1}^a+n_{l+1}^m)}$$

n_l^a addition nodes
 n_l^m multiplication nodes
 $\odot$ element-wise multiplication



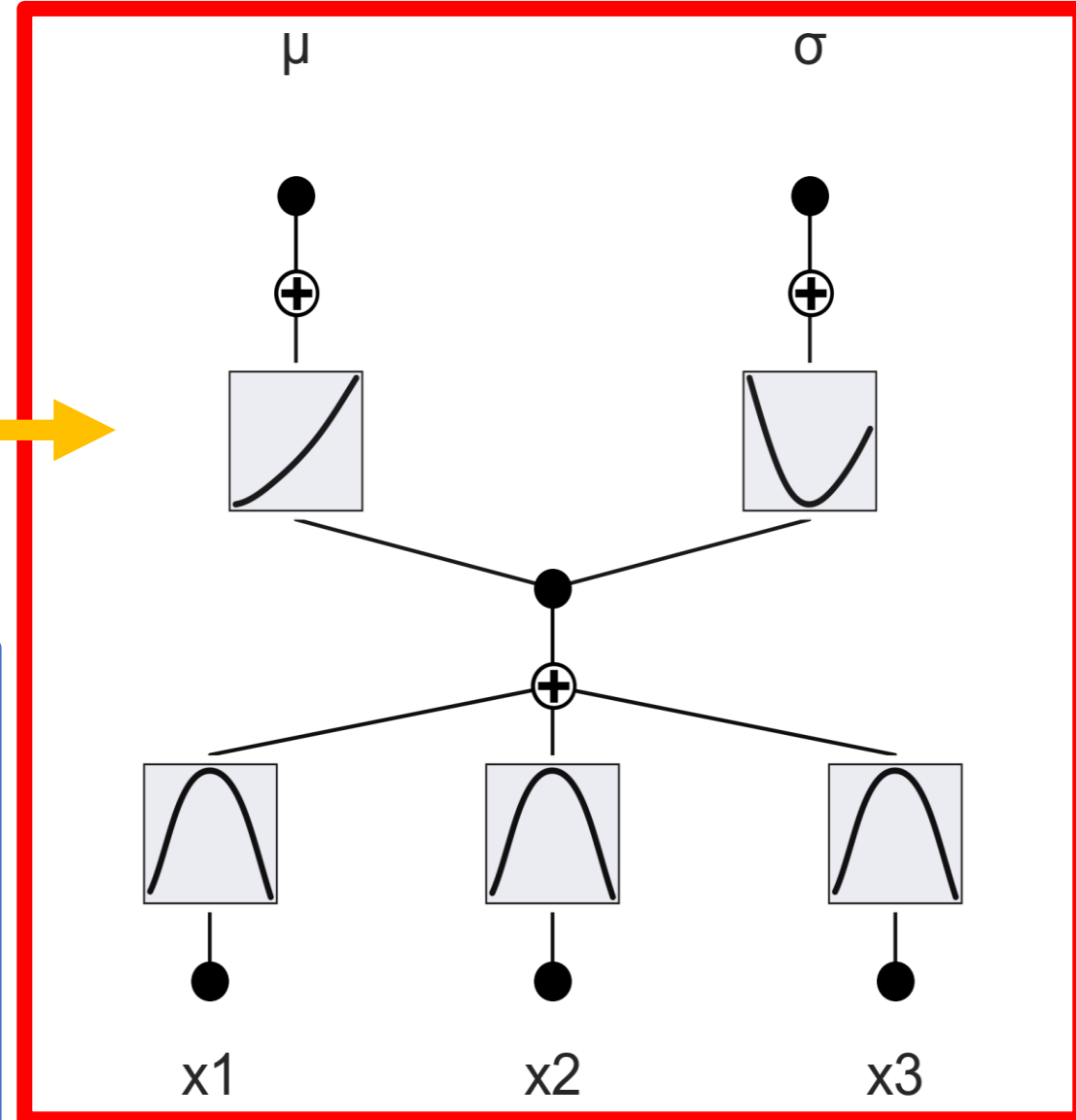
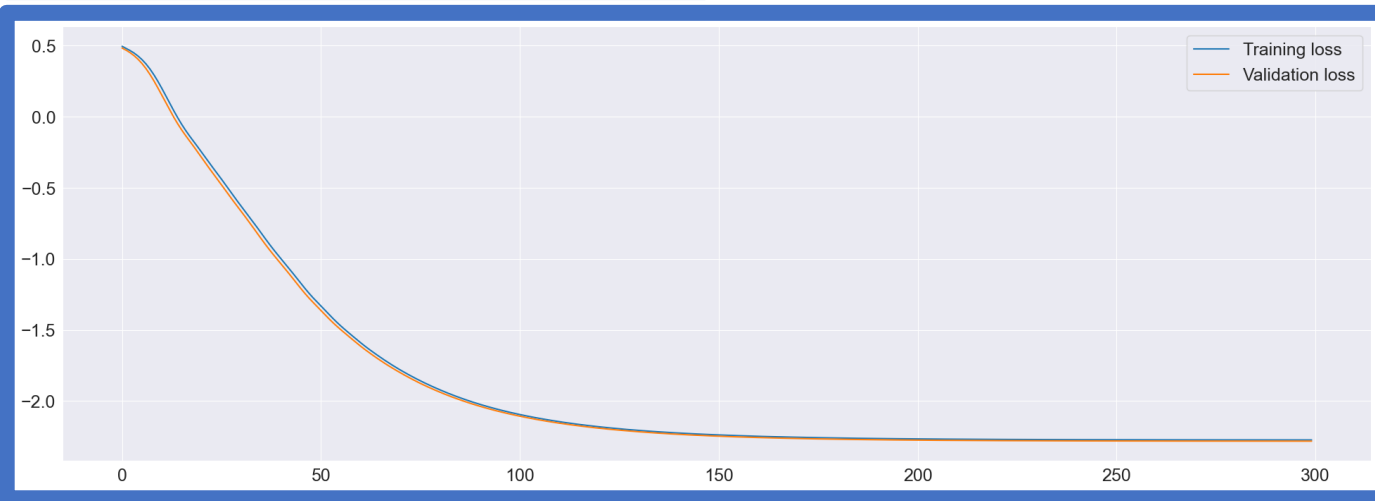
- **Auxiliary variables** can be added to reduce the KAN architecture to a more interpretable one.
- **Sparse initialization** is used to mimic the sparse connections of common formulae.
- **Hypothesis Testing** is easy to use. MultKAN automatically saves checkpoints for rewinding.

Can they really be useful?



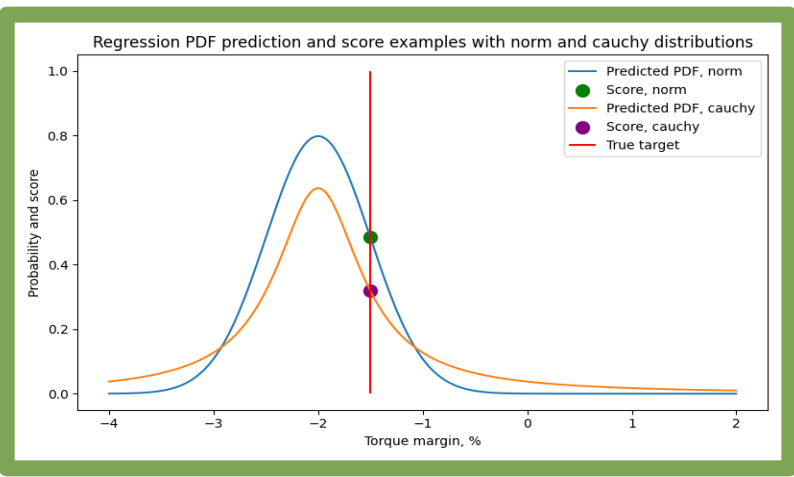
□ To assess their ability to solve a probabilistic regression task, we trained a **[3, 1, 2]** PyKAN on a standard 3D normal PDF.

□ The resulting network is **very easy to interpret.**



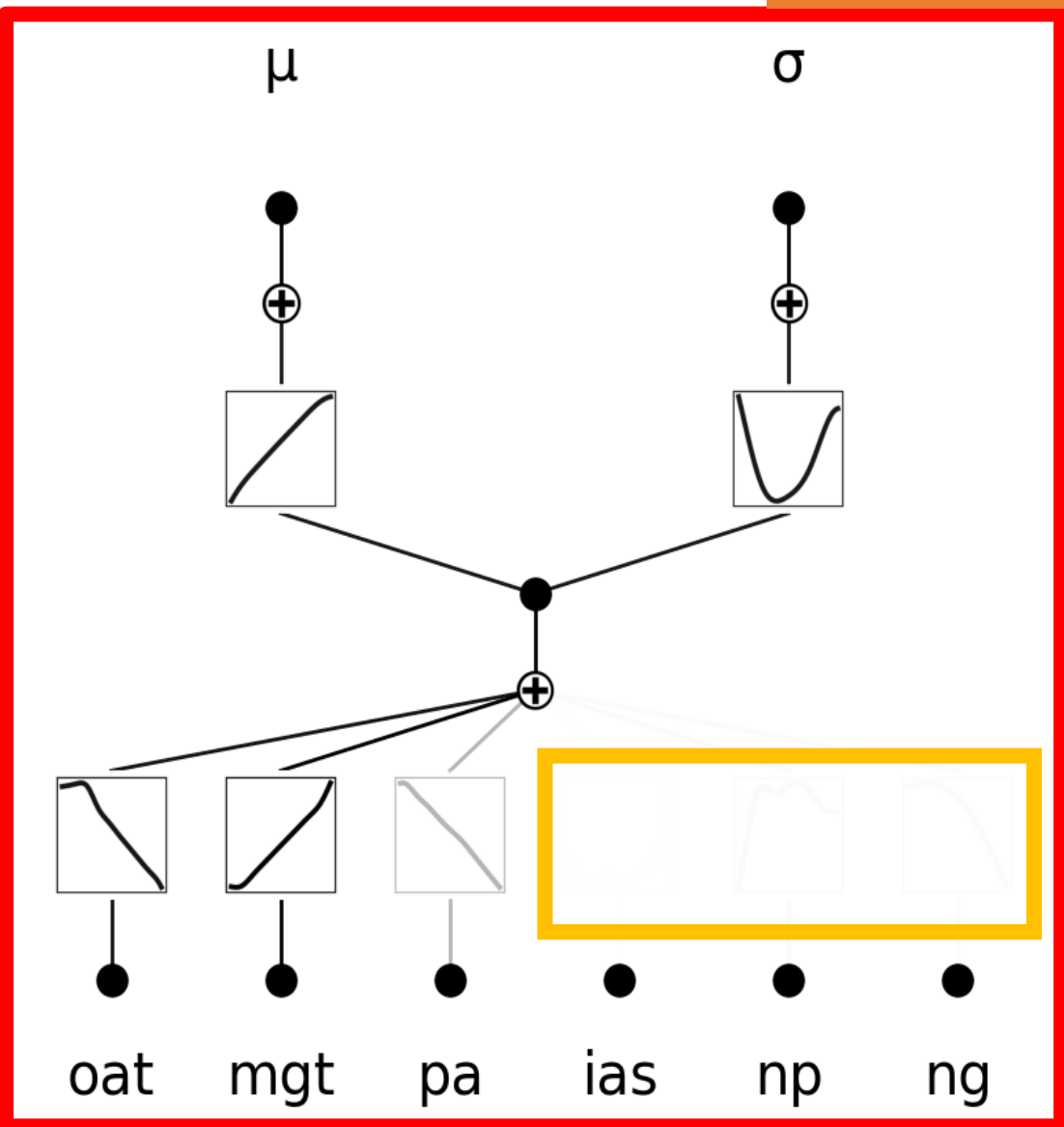
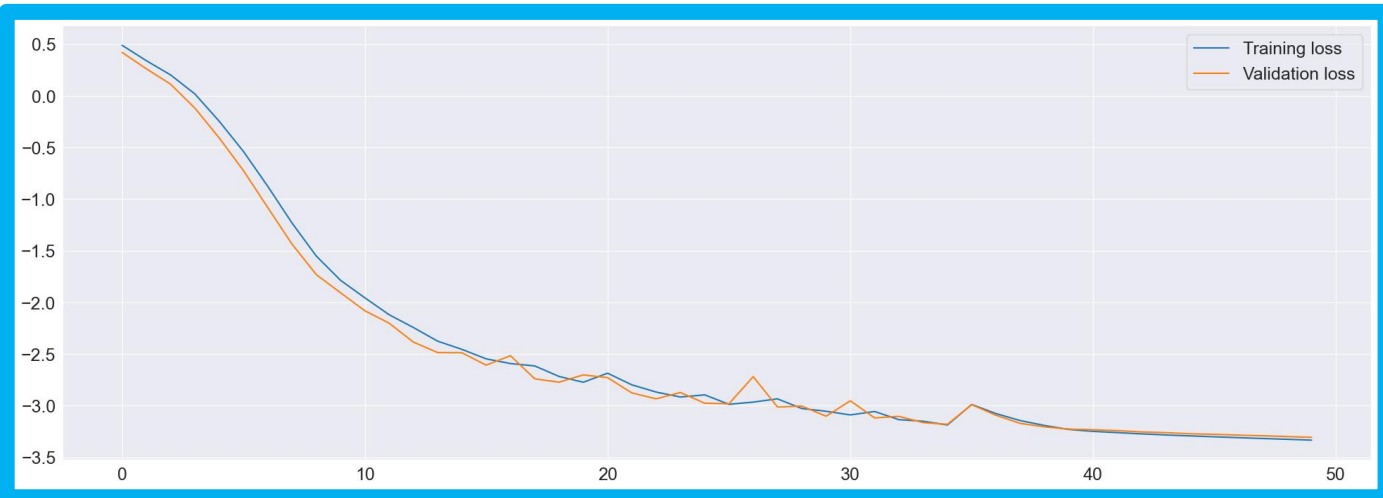
Probabilistic Regression with PyKAN

FLOPs: 93
Params: 60



❑ We said we would consider the PDF
Gaussian ⇒ **NLL**
needs to be employed over the classic MSE.

$$NLL(\mu, \sigma, \bar{y}) = -\ln \left(\frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{\bar{y}-\mu}{\sigma}\right)^2} \right) = \frac{1}{2} \ln(2\pi\sigma^2) + \frac{(\bar{y} - \mu)^2}{2\sigma^2}$$

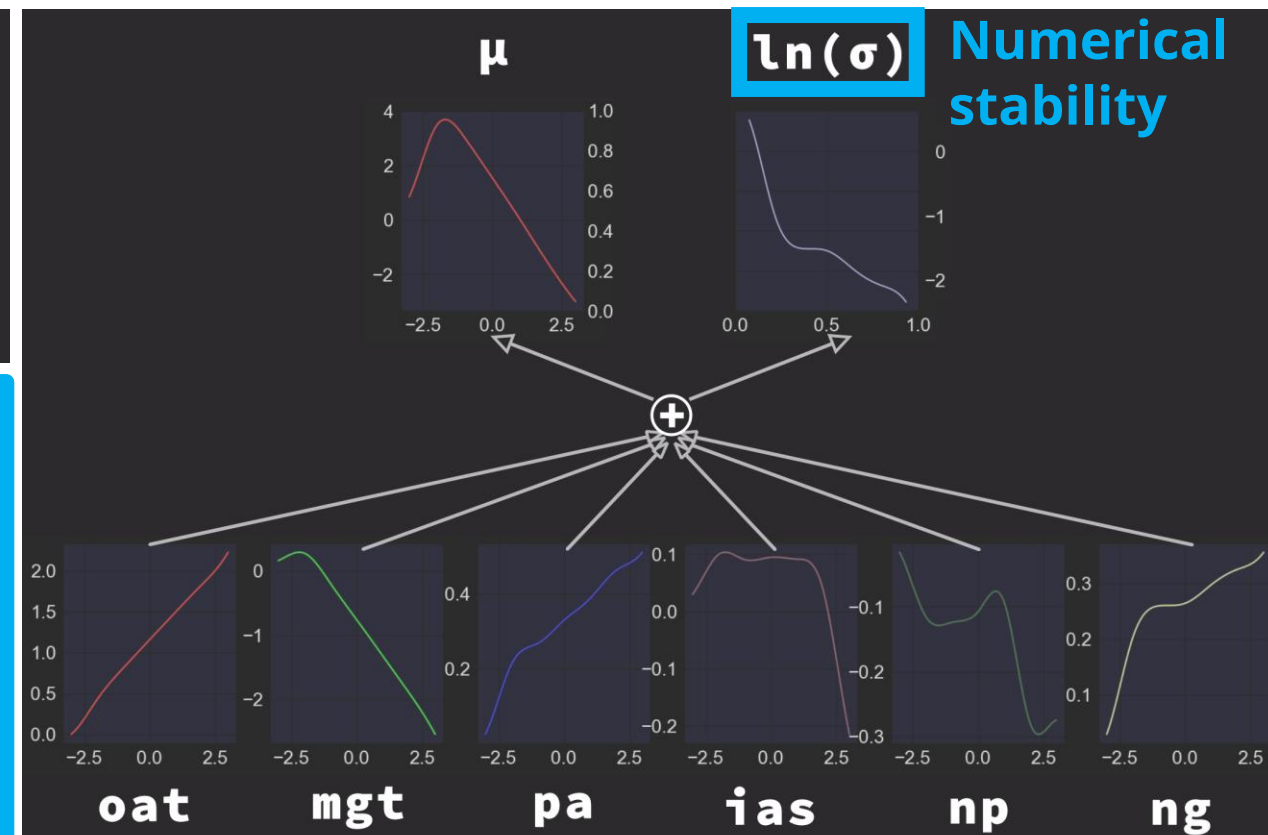
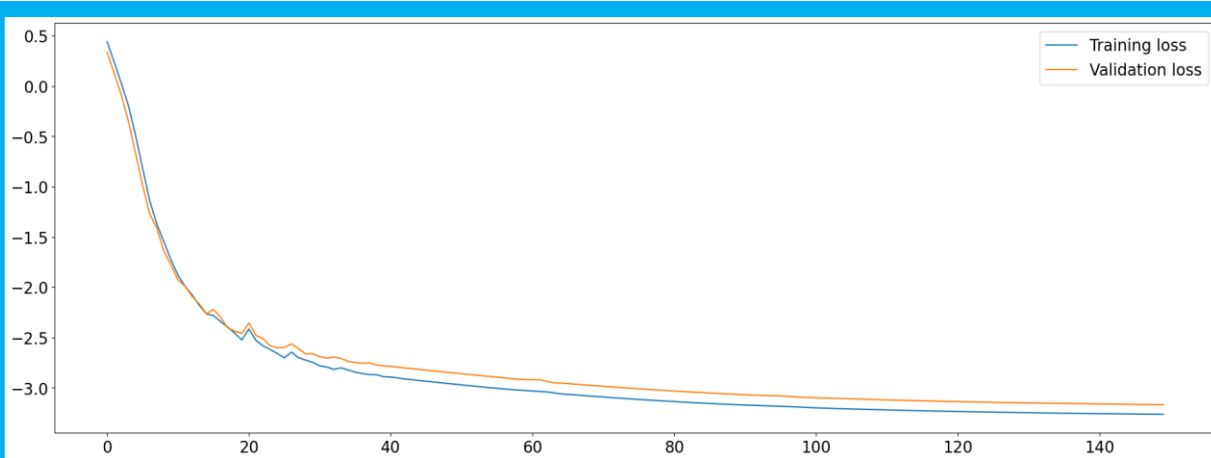


Probabilistic Regression with EfficientKAN

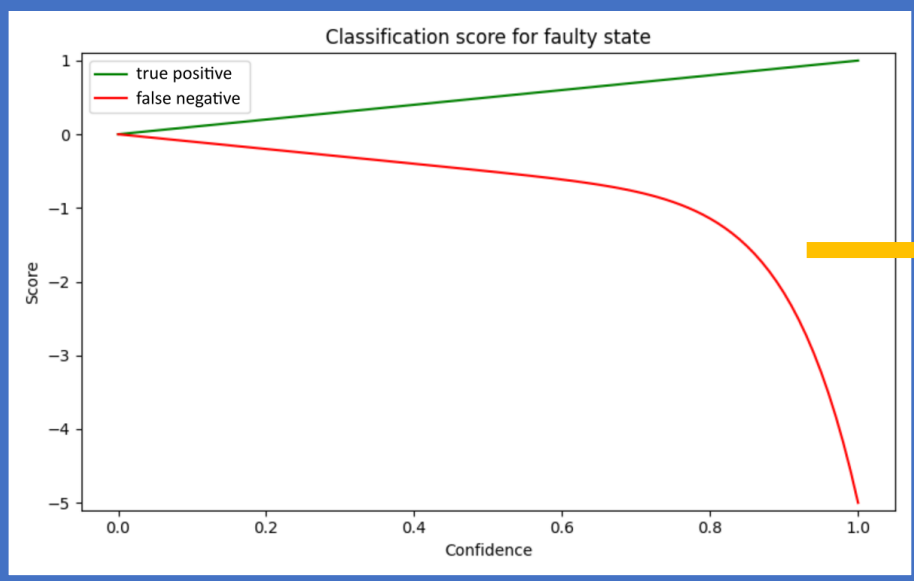
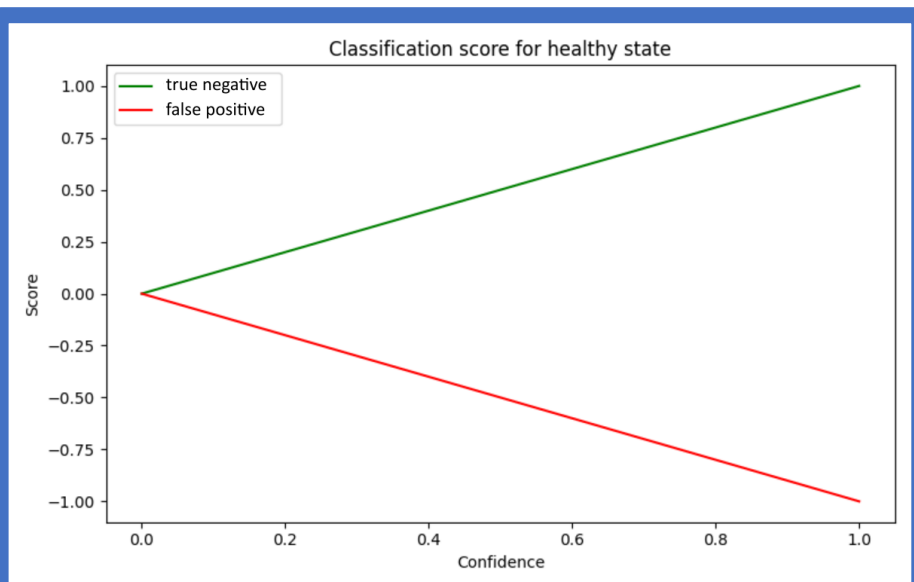


- ❑ *EfficientKAN* is an **optimized implementation of KAN**. In contrast to *PyKAN*, however, it **lacks some features**. Plotting is one of them.
- ❑ Here we can see that it reaches almost the same accuracy than the *PyKAN*, but with a considerably faster training — **5 times less operations, way less memory usage.**

```
base_output = F.linear(self.base_activation(x), self.base_weight)
spline_output = F.linear(
    self.b_splines(x).view(x.size(0), -1),
    self.scaled_spline_weight.view(self.out_features, -1),
)
output = base_output + spline_output
```



Classification scoring



- The challenge defines the following *scoring function*.

if ($confidence < 0$) or ($confidence > 1$), then $score = -100$

if $pred_label \notin \{0, 1\}$, then $score = -100$

$$confidence = \begin{cases} -confidence & \text{if } pred_label \neq true_label \\ confidence & \text{if } pred_label = true_label \end{cases}$$

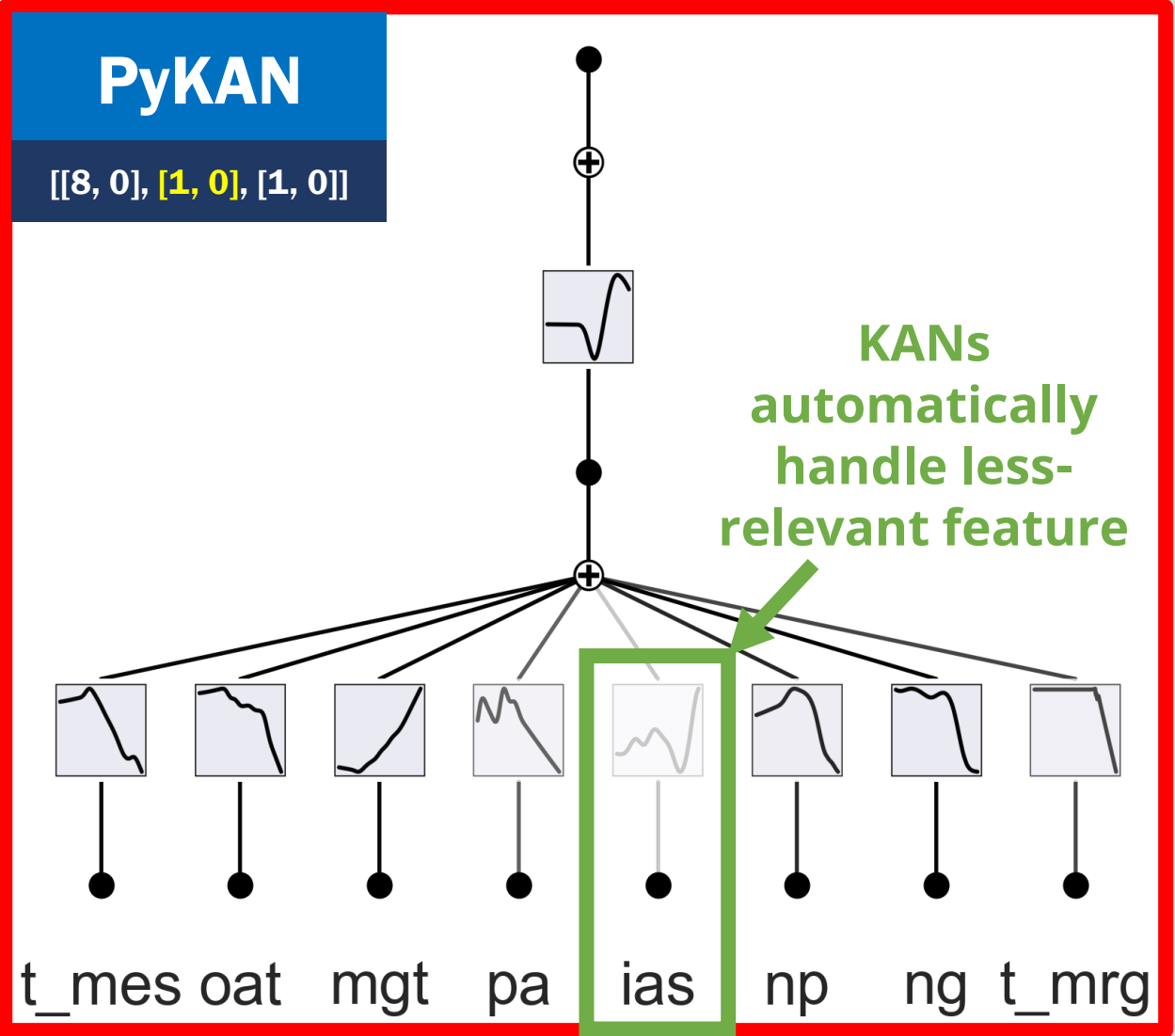
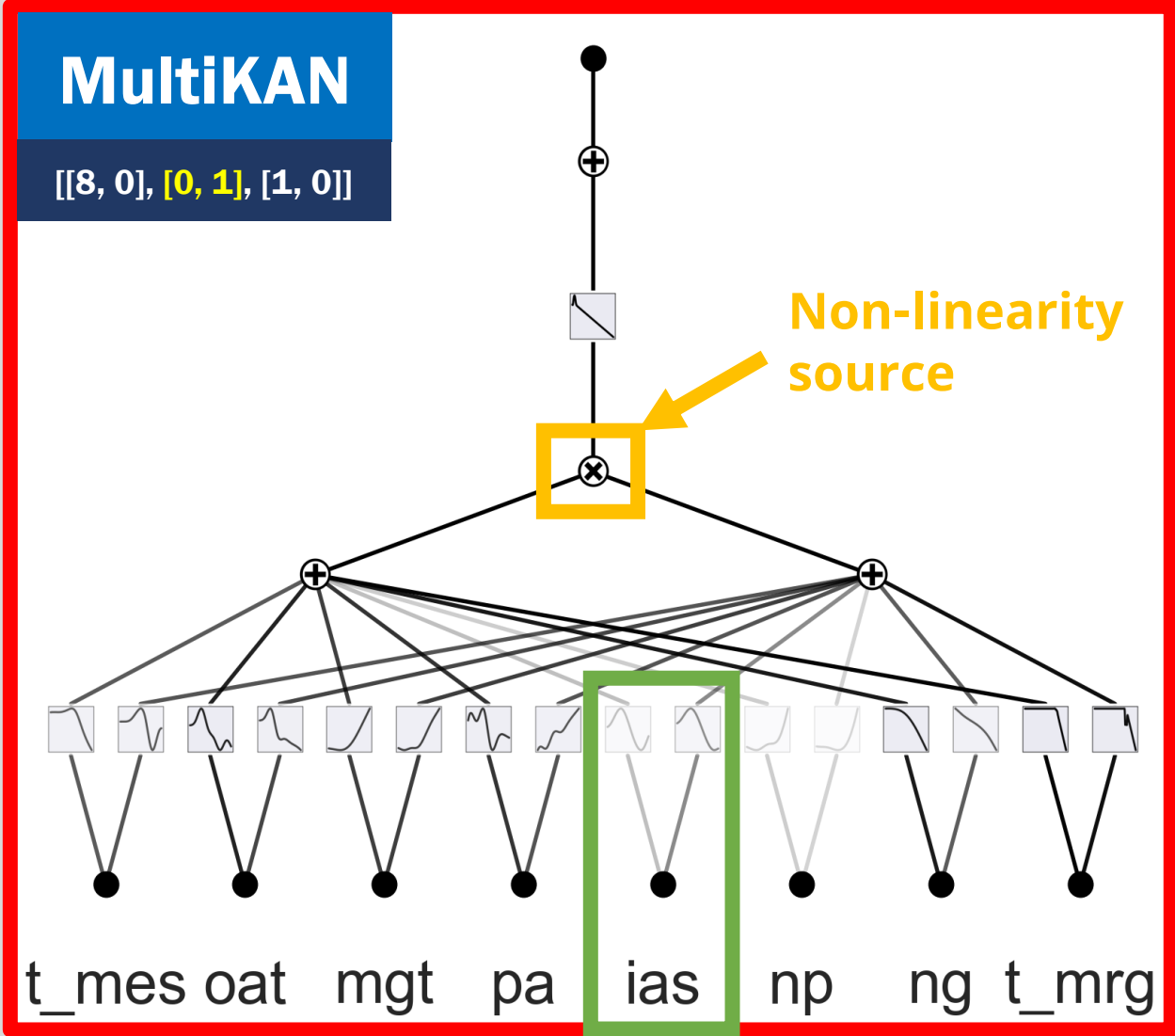
$$score = \begin{cases} confidence & \text{if } true_label = 0 \\ confidence & \text{if } true_label = 1 \text{ and } confidence \geq 0 \\ 4 \times confidence^{11} + confidence & \text{if } true_label = 1 \text{ and } confidence < 0 \end{cases}$$

- This custom metric is critical because **reducing false negatives** (not detecting a failure) is more important than reducing false positives

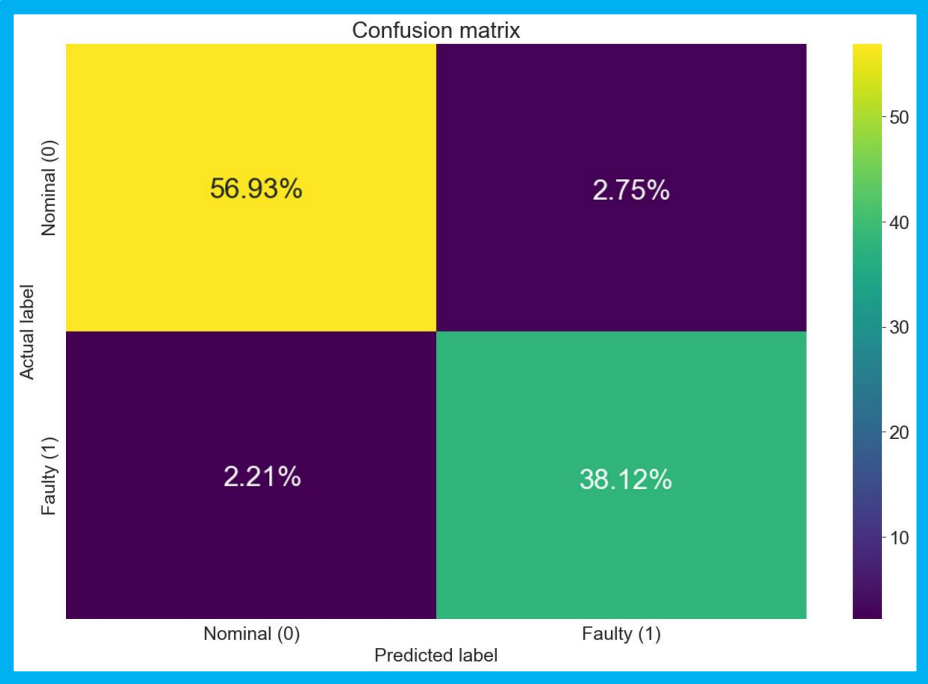
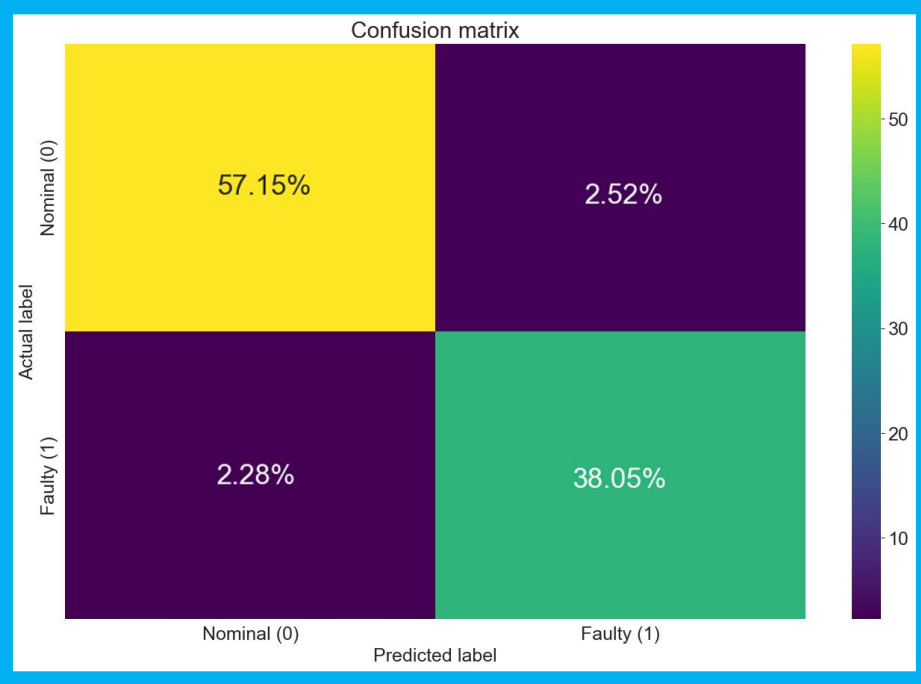
We assume that the inference is not made so often, so that the effect of the missed alarm is short-lived.



Fault Detection with PyKAN



Results of Fault Detection with PyKAN



Model	MultiKAN	PyKAN
Avg Score	0.819	0.815
Accuracy	0.952	0.950
Precision	0.938	0.933
Recall	0.944	0.945
F1-score	0.941	0.939

PHM North America 2024 Challenge



EDA and Preprocessing

What kind of data
will we be working
with?



Probabilistic regression and fault detection

What are the results of
our experiments?



Next steps

What's
missing?

Next steps

In the light of these results, the following issues need to be addressed:

- ❑ At inference time, the **output and the inputs of the regressor should be denormalized.**
- ❑ The **classifier should use the estimated torque target as a feature.**
- ❑ The **selected features should be used** in both regression and classification to increase the accuracy.
- ❑ As discussed in PCA, the ratio ng/np could be used to **train 2 classifiers in parallel.**
- ❑ **A qualitative test should be carried out on the unlabeled test set,** since it is generated from different engines.